Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster en Ingeniería Matemática y Computación

Aplicación de la curva de Hilbert bidimensional a la transferencia térmica

Trabajo Fin de Estudios

presentado por: Ivan Alfredo Rodrigo Arbós

Tipo de trabajo: Aplicación práctica

Dirigido por: Dr. Aleix Alvarez

Director UNIR: Dr. Íñigo Sarría Martínez de Mendivil

Ciudad: Barcelona, Catalunya

Fecha: Febrero de 2026

Agradecimientos

Primero que todo, agradecer al tribunal por dedicar un tiempo en analizar este trabajo y valorar este pequeño trozo de mí. También a cualquier lector que se haya interesado por este increíble campo.
Agradecer a la **UNIR** la increíble gestión durante todo el año, ha sido un máster increíble. Pero debo destacar la gran labor del director del trabajo **Íñigo Sarría Martínez**. Cogió las riendas de este estudio cuando todo estaba en marcha y en nada se adentró en el mundo y consiguió mantener el ritmo claro y preciso que requería la dirección. Íñigo, haría cualquier investigación contigo, gracias.

A la **Laura Costa**, mi profesora de matemáticas en la Universidad de Barcelona que, aun habiendo terminado mi etapa allí como estudiante, nunca ha dejado de ayudarme a ser mejor matemático cada día. Ojalá llegar a ser una pequeña parte de lo profesional que eres.

Seguidamente a mi familia matemática del día a día, el **departamento de la ETSAB**. Pere, Santos, Germán, Álvaro, Pol, Marina, Pau. Hacéis que me enamore cada día más de las matemáticas. Aprendo y "xalo" rodeado de esta ciencia y sin vosotros no sería posible. Pero no puedo olvidar a **Aleix Álvarez**, el director y motor principal de este trabajo que merece mención especial. Eres un profesional excelente dentro y fuera del trabajo. Esto solo ha hecho más que empezar y deseo seguir trabajando junto a ti mucho más tiempo, ya que sabemos que este estudio aún no se ha terminado.

Momento de dar gracias a la gente externa del ámbito científico por su ayuda durante cada día de mi vida. Amigos, en general, gracias porque siempre sabéis hacerme sentir como en casa allí donde esté.
Pero hay que destacar por su gran personalidad a varias personas:

De Barcelona, **Jhonan y Jordan Cabrera**, mis hermanos del baloncesto, que de compartir una afición de deporte ha nacido una gran amistad. Ojalá no perderos nunca.

De mi pueblo destacar a mis grupos "Fem lo que podem", "Supermam" y "TAUxSM". Pero en particular a **Abelló, Queral, Roger, Castellá, Fonsi, Eudald, Arnau y Pol**. Seguís desde siempre, día a día, a mi lado demostrando que la amistad es algo para toda la vida.

Carlos Sabaté, por tener una confianza en mí plena y estar presente en todo momento, bueno o malo, ayudando como nadie en el mundo sabe.

Álvaro Luque, Joan Guich y Onofre Bisbal, lo que unió la Universidad no lo va a separar nadie. He encontrado gracias a las matemáticas a mis mejores amigos y hermanos. Nunca habéis dejado de ayudarme, sabiéndome aconsejar en cada situación. Sois los mejores.

Laia Figueras y Gemma "Gemmeta" Roig, chicas que están a mi lado cada día desde que soy pequeño y nunca han dejado de hacerme sonreír, sea cual sea la situación. Ojalá todo el mundo tuviera amigas como vosotras.

Y finalmente mi familia, que se preocupa por mí siempre y busca que esté lo mejor posible.
Mi prima **Judith Solé** y mi tío **Jordi**, que pase lo que pase están ahí demostrando que la familia no va a fallar nunca.

La **familia Massó Baguena**, 5 magníficos seres porque uno tiene 4 patas, ladra y chupa. Me han abierto las puertas de su casa y me han tratado como si fuera uno más desde el primer momento. Habéis sido muy especiales para mí en estos últimos meses tan importantes y exigentes. Sois flipantes.

Y dar mis últimos agradecimientos a las personas más importantes para mí.

Mis padres, **Margarita y Gonzalo**, que dan todo lo que tienen y más por mí. Hacen que nunca me falte de nada y luchan tanto como yo por mis objetivos. Sin vosotros no hubiera conseguido ni la mitad de todo lo que tengo. Sois los mejores padres del mundo, las personas que mejor me entienden y la mejor pareja que conozco. Sois el mejor ejemplo que pueda existir. Os quiero mucho, papis.

Finalmente a **Judit Massó Giménez**, una personita increíble que ha aparecido en la etapa final de este largo año tan importante. No tan solo me ha conocido al detalle y lo sigue haciendo, sino que me ha escuchado, entendido y valorado como nadie antes lo había hecho. Eres oro. En esta etapa final tan difícil ha estado a mi lado en todas las complicaciones, ayudándome a llevarlas de la mejor manera.
Ojalá no perderte nunca porque soñaba con una princesa y eres la demostración de que los sueños se cumplen y las películas de Disney existen.

Gracias a todos por todo y más.

Índice de Contenidos

Agradecimientos I

Resumen IX

Abstract X

1. Introducción 1

1.1. Motivación 1

1.1.1. Propiedades de la curva de Hilbert 4

1.2. Planteamiento del trabajo 5

1.3. Estructura del trabajo 6

2. Contexto y Estado del Arte 8

2.1. Mejorar la transferencia térmica a través de micro-geometria 8

2.2. Intensificar la transferencia térmica con hendiduras 9

2.3. Modificaciones estructurales en transferencia térmica 10

2.4. Estudio aplicado sobre la geometría del recorrido 12

2.5. Conclusión 14

3. Identificación de requisitos 16

3.1. Metodología 16

3.2. Software 16

4. Objetivos 17

4.1. Objetivo principal 17

4.2. Objetivos específicos 17

5. Curva que rellena el espacio 18

6. Transferencia de calor 21

6.1. Conducción 22

6.2. Convección 24

6.3. Radiación 25

7. La Curva de Hilbert **28**

7.1. Definición formal 28

7.2. Propiedades 29

7.3. Definición y codificación personal 30

7.4. Código para la implementación 32

7.5. Conclusión 39

8. Transferencia térmica sobre la curva de Hilbert **40**

8.1. Transferencia térmica en el punto p_i 41

8.1.1. Balance de energía 43

8.1.2. Convección interna 43

8.1.3. Convección externa 44

8.1.4. Conducción 45

8.1.5. Radiación 46

8.2. Recurrencia de la transferencia térmica 47

8.2.1. Determinación de los parámetros del modelo 48

9. Codificación **51**

9.1. Dibujar temperatura 51

9.2. Codificar los términos de la recurrencia 52

9.3. Codificación del algoritmo principal 53

9.4. Tubería de serpentín 55

10. Resultados **56**

10.1. Conclusión global 59

11. Conclusiones y trabajo futuro **61**

11.1. Trabajo futuro 64

Referencias 65

A. Apéndice **66**

A.1. Código para simulación 66

A.1.1. Librerías 66

A.1.2. Recorrido de la tubería 66

A.1.3. Dibujar la curva 71

A.1.4. Crear tubería con temperatura 72
A.1.5. Representar la temperatura sobre la superficie 72
A.1.6. Términos de la recurrencia 74
A.1.7. Encontrar vecinos no adyacentes 75
A.1.8. Algoritmo principal 75
A.2. Tubería serpentín 77
A.3. Generar video de simulación 78
A.4. Codificar paredes de la tubería 83
A.5. Libro de arte 94

Índice de Ilustraciones

1.1. Transferencia térmica en serpentina en condensador y suelo radiante. (Rodrigo Arbós. 2025) 1
1.2. Curva de dragón. Fuente: https://www.mathcurve.com/fractals/fractals.shtml 2
1.3. Curva de Gosper. Fuente: https://www.mathcurve.com/fractals/fractals.shtml 3
1.4. Curva de Peano. Fuente: https://www.mathcurve.com/fractals/fractals.shtml 3
1.5. Curva de Hilbert. Fuente: https://www.mathcurve.com/fractals/fractals.shtml 3

6.1. Transferencia térmica del fluido por una tubería. (Rodrigo Arbós. 2026) . . 22
6.2. Transferencia térmica por conducción entre tuberías. (Rodrigo Arbós. 2026) 23
6.3. Transferencia térmica por convección por una tubería. (Rodrigo Arbós. 2026) 25
6.4. Transferencia térmica por radiación entre superficies. (Rodrigo Arbós. 2026) 27

7.1. Mapeando el cuadrado. (Sagan, 1994) 29
7.3. Primera, segunda, tercera y cuarta iteración de la curva de Hilbert. (Rodrigo Arbós. 2025) 31
7.4. Transformación de la curva dentro del cuadrado según su orientación. (Rodrigo Arbós. 2026) 31
7.5. División y identificación de los subcuadrados para una nueva iteración. (Rodrigo Arbós. 2026) 32
7.6. Centros del rectángulo (1,1) con iteración 1 33
7.7. Centros del rectángulo (1,1) con iteración 2 33
7.8. Representación de los ejemplos 7.6, 7.7 (Rodrigo Arbós. 2025) 34
7.9. Representación con computación de las últimas dos figuras de 7.3 (Rodrigo Arbós. 2025) 34
7.10. Curva de Hilbert sobre rectángulo (3,1) y 2 iteraciones. (Rodrigo Arbós. 2025) 35
7.11. Curva de Hilbert sobre rectángulo (3,1) y 3 iteraciones. (Rodrigo Arbós. 2025) 35
7.12. Curva de Hilbert sobre rectángulo (3,1) y 4 iteraciones. (Rodrigo Arbós. 2025) 36
7.13. Curva de Hilbert sobre rectángulo (1,1) y 6 iteraciones. (Rodrigo Arbós. 2025) 37
7.14. Curva de Hilbert sobre rectángulo (1,3). (Rodrigo Arbós. 2025) 38

8.1. Variables que determinan el canal del fluido. (Rodrigo Arbós. 2026) 42

8.2. Tubería de Hilbert con estructura rectangular. (Rodrigo Arbós. 2026) . . . 42
8.3. Esquema de la convección interna en la simulación. (Rodrigo Arbós. 2026) 44
8.4. Esquema de la convección externa en la simulación. (Rodrigo Arbós. 2026) 44
8.5. Esquema de la conducción en la simulación. (Rodrigo Arbós. 2026) 45
8.6. Esquema de la radiación en la simulación. (Rodrigo Arbós. 2026) 46

9.1. Situación inicial de la temperatura de los nodos. (Rodrigo Arbós. 2026) . . 52
9.2. Situación inicial sobre toda la superficie. (Rodrigo Arbós. 2026) 52
9.3. Representación de vecinos. (Rodrigo Arbós. 2026) 53
9.4. Representación de vecinos. (Rodrigo Arbós. 2026) 53
9.5. Transporte de masa por Hilbert con iteración 4. (Rodrigo Arbós. 2026) . . . 54
9.6. Resultado de la simulación sin perdida de calor. (Rodrigo Arbós. 2026) . . . 54
9.7. Curva con geometria serpentín en rectangulo 1x1. (Rodrigo Arbós. 2026) . . 55
9.8. Transporte de masa por Serpentín con iteración 4. (Rodrigo Arbós. 2026) . 55

10.1. Equilibrio térmico en rectángulo 1×1. (Rodrigo Arbós. 2026) 56
10.2. Equilibrio térmico en rectángulo 1×1. (Rodrigo Arbós. 2026) 57
10.3. Equilibrio térmico en rectángulo 1×1. (Rodrigo Arbós. 2026) 58
10.4. Simulación de la transferencia térmica en rectángulo 1×1. (Rodrigo Arbós. 2026) 58

A.1. Generar tubería de Hilbert bidimensional 93
A.2. Tubería cilíndrica. (Rodrigo Arbós. 2026) 94
A.3. Tubería rectangular. (Rodrigo Arbós. 2026) 94
A.4. Separación de una superficie por bloques. (Rodrigo Arbós. 2026) 95

Índice de Tablas

1.1. Comparación de propiedades geométricas entre las curvas que rellenan el espacio. 5

2.1. Comparación entre estudios sobre la mejora de transferencia térmica 9

2.2. Comparación entre estudios sobre intensificación de transferencia térmica . 10

2.3. Comparación entre estudios sobre mejoras de transferencia térmica estructurales 12

2.4. Comparación sobre diseños geométricos del recorrido 13

2.5. Resumen comparativo de enfoques para la mejora de la transferencia térmica 15

8.1. Coeficientes de convección en transferencia térmica. 49

Resumen

Este trabajo aborda el campo científico de la física de la mecánica de fluidos en relación con la transferencia térmica, el campo matemático de la geometría fractal y la simulación computacional. El objetivo principal es comparar la geometría de curva serpenteante convencional de los paneles radiantes con la curva de Hilbert en la transferencia de calor. Metodológicamente, en la primera parte del trabajo se desarrolla la base teórica de cada campo de trabajo para posteriormente formalizar la razón que combina todos los conceptos y crear una recurrencia que define el comportamiento del problema abordado. Finalmente, se codifica esta relación recursiva para poder recrear las simulaciones, analizar cada situación y compararlas. Los resultados no demuestran que la nueva geometría aporte una mejora significativa en cuanto a la temperatura del equilibrio térmico. Mediante cálculo computacional se ve que hay una posible mejora del enfriamiento, pero no se simulan todos los parámetros que aporta la curva de Hilbert y que podrían aumentar esta eficiencia térmica. En el análisis final sí destaca la nueva distribución térmica sobre la superficie, factor que va ligado a las propiedades de la curva fractal y demuestra el gran potencial de esta geometría en el campo. Este resultado permite valorar las posibles aplicaciones en otras situaciones y sacarle provecho a la geometría y simulación diseñada.

Palabras clave: Simulación computacional, Transferencia térmica, curva de Hilbert, Disipador de calor.

Abstract

This work addresses the scientific field of fluid mechanics in relation to heat transfer, the mathematical field of fractal geometry, and computational simulation. The main objective is to compare the conventional serpentine geometry of radiant panels with the Hilbert curve in heat transfer. Methodologically, in the first part of the work, the theoretical foundations of each field are developed in order to subsequently formalize the rationale that combines all the concepts and to create a recurrence that defines the behavior of the problem addressed. Finally, this recursive relationship is implemented to recreate the simulations, analyze each situation, and compare them. The results do not demonstrate that the new geometry provides a significant improvement in terms of thermal equilibrium temperature. Through computational calculations, a possible improvement in cooling can be observed, but not all the parameters introduced by the Hilbert curve that could increase this thermal efficiency are simulated. In the final analysis, the new thermal distribution over the surface stands out, a factor that is linked to the properties of the fractal curve and demonstrates the great potential of this geometry in the field. This result makes it possible to assess potential applications in other situations and to take advantage of the designed geometry and simulation.

Keywords: Computational simulation, Heat transfer, Hilbert curve, Heat sink.

1. Introducción

¿Alguna vez te has fijado en el condensador de una nevera o en las tuberías del suelo radiante?
Son sistemas que emplean conductos dispuestos en forma serpenteante que también se encuentran en numerosos electrodomésticos y equipos de climatización. Su objetivo es disipar el calor o favorecer la transferencia térmica de manera más uniforme a lo largo de una superficie.

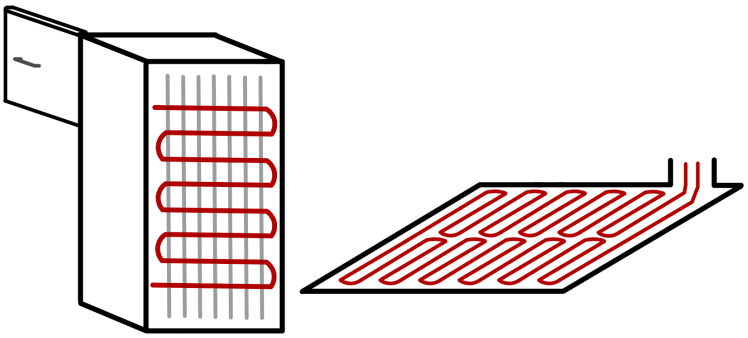


Figura 1.1: Transferencia térmica en serpentina en condensador y suelo radiante. (Rodrigo Arbós. 2025)

Pero, ¿es realmente ésta la geometría más eficiente para lograr una distribución óptima de la transferencia de calor?

1.1. Motivación

La idea de esta investigación es determinar si existe una curva plana que mejore la transferencia térmica de la geometría en serpentina sobre una superficie también plana.

Debido a la gran diversidad de formas que se pueden definir sobre una superficie, se ha decidido estudiar **la curva de Hilbert**. Esta geometría no ha sido escogida únicamente por ser una topología utilizada en otros campos de la ingeniería. La curva de Hilbert presenta múltiples propiedades que, en conjunto, la hacen especialmente interesante y, al no ser trivial de reproducir, supone una novedad en el campo de estudio.

Si se vuelve al origen de la selección de esta topología, el objetivo es definir una geometría que permita cubrir el espacio de manera similar a la serpentina. A partir de esta idea surge el estudio de curvas fractales y, en particular, de aquellas capaces de rellenar el

espacio.

En una primera lectura de textos como (Sánchez, 2023) o (Mandelbrot, 1982), se introduce el **conjunto de Cantor**, elemento que se define a partir de un proceso iterativo de subdivisión. Este tipo de construcciones da lugar a estructuras que presentan un comportamiento similar: las **estructuras autosemejantes**. Dichas estructuras se generan mediante la repetición de una misma forma a distintas escalas.

En estos textos también se estudia la dimensión de las figuras, en particular la denominada **dimensión de Hausdorff**. Esta dimensión no mide la dimensión topológica de la curva, que claramente es igual a 1, sino que mide la dimensión del conjunto que genera. Dado que se busca una estructura capaz de cubrir una superficie, como por ejemplo el suelo de una habitación o el panel del fondo de un frigorífico, para este estudio se consideran aquellas geometrías cuya dimensión de Hausdorff es dos.
Es en este marco donde aparecen las denominadas **curvas que rellenan el espacio**, formalizadas como aplicaciones continuas cuya imagen cubre completamente la región del plano sobre la que están definidas y que son estructuras autosemejantes.

Con estas condiciones, existe una infinidad de curvas planas, entre las que se pueden destacar, a modo de ejemplo:

1. **Curva de dragón**

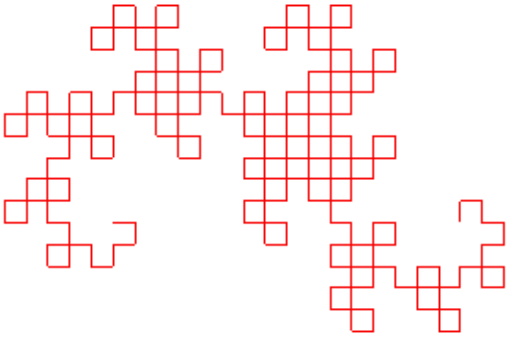


Figura 1.2: Curva de dragón. Fuente: https://www.mathcurve.com/fractals/fractals.shtml

2. **Curva de Gosper**

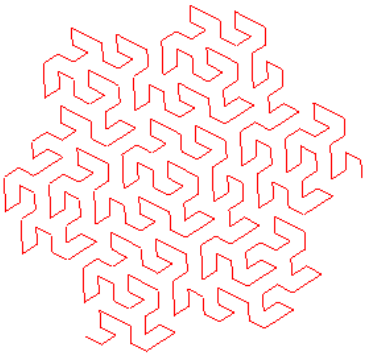
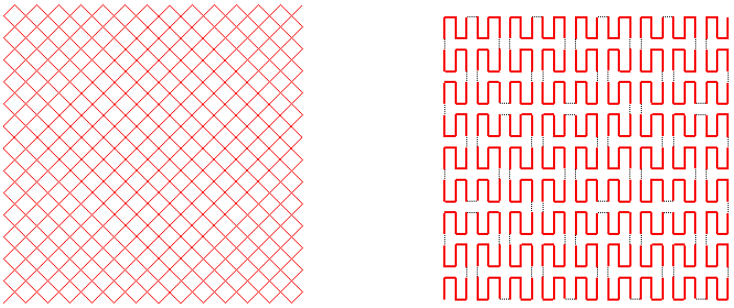


Figura 1.3: Curva de Gosper. Fuente: https://www.mathcurve.com/fractals/fractals.shtml

3. **Curva de Peano**



(a) Forma general (b) Forma clásica

Figura 1.4: Curva de Peano. Fuente: https://www.mathcurve.com/fractals/fractals.shtml

4. **Curva de Hilbert**

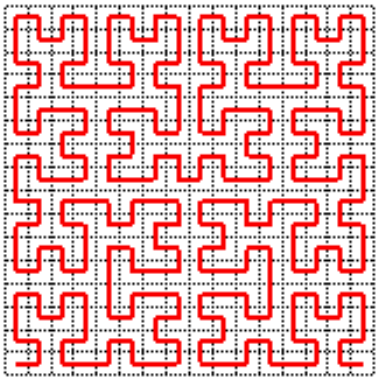


Figura 1.5: Curva de Hilbert. Fuente: https://www.mathcurve.com/fractals/fractals.shtml

Y si se toman como referencia estas curvas y se analizan con propiedades estudiadas en (Hilbert, 1891) o (Sagan, 1994), es decir, con características intrínsecas a las mismas, se puede ver cuál es la más adecuada para una posible geometría capaz de reinventar la transferencia térmica.

1.1.1. Propiedades de la curva de Hilbert

1. **Continuidad**

   Se trata de una propiedad indispensable en este estudio y que cumplen todas las opciones consideradas, ya que cada una de ellas se construye como límite de iteraciones sobre trayectorias sin discontinuidad.

2. **Rellenar el espacio**

   Se trata de aplicaciones continuas sobreyectivas del intervalo unidad en una región bidimensional.

3. **Construcción iterativa y autosimilitud**

   Propiedad presente en cada una de las geometrías presentadas y que es una característica fundamental de los objetos fractales. Cada iteración consiste en una transformación reescalada de la estructura anterior que permite generar configuraciones complejas a partir de reglas simples.

4. **Uniformidad espacial**

   Tanto la curva de Hilbert como la de Peano presentan una distribución homogénea sobre la superficie rectangular, es decir, visitan todas las regiones de manera equilibrada. Las curvas del dragón y de Gosper, aunque cubren regiones amplias, no presentan una uniformidad tan regular en la distribución espacial.

5. **Preservación de la localidad**

   Es una propiedad que explica que puntos cercanos en el parámetro tienden a corresponderse con puntos cercanos en el plano. La curva de Peano presenta una preservación de la localidad más limitada, mientras que la curva del dragón y la de Gosper muestran un comportamiento menos favorable en este sentido. Esta es la característica de la curva de Hilbert por excelencia; destaca frente a las demás, lo que la convierte en la candidata a resolver el problema de este trabajo.

A continuación, se presenta un resumen de las propiedades en forma de tabla, donde se recogen las características de cada una de las curvas estudiadas:

Tabla 1.1: Comparación de propiedades geométricas entre las curvas que rellenan el espacio.

Propiedad	**Hilbert**	**Peano**	**Dragón**	**Gosper**
Continuidad	Sí	Sí	Sí	Sí
Rellenar el espacio	Sí	Sí	No	No
Construcción iterativa y autosimilitud	Sí	Sí	Sí	Sí
Uniformidad espacial	Alta	Alta	Media	Media-alta
Preservación de la localidad	Alta	Media	Baja	Media

Aunque se podría estudiar la curva de Peano como posible solución al problema, se ha descartado porque, como se observa en la figura 1.4b, presenta una pequeña similitud en las primeras iteraciones con la serpentina, y la idea es crear una geometría nueva no trivial.

En conclusión, aunque estas curvas comparten propiedades fractales como la autosimilitud y la continuidad, la curva de Hilbert destaca por combinar también la propiedad de rellenar el espacio con una muy buena preservación de la localidad y una distribución espacial uniforme. Se considera, por tanto, una buena candidata para encontrar una mejora en este campo, tal y como ha ocurrido en otros ámbitos de la ingeniería, como la indexación espacial, el procesamiento de imágenes o los sistemas de información geográfica.

1.2. Planteamiento del trabajo

1. **Teoría**

   Es la primera parte de este trabajo de fin de máster (TFM). Forma los primeros capítulos de la investigación y en ella se desarrollan los conceptos y se definen todos los elementos necesarios para comprender cada una de las partes posteriores.

2. **Desarrollo práctico**

   Aquí se utilizan las leyes definidas anteriormente para crear la base práctica y definir

los métodos que permiten implementar los cálculos necesarios para comprobar si la propuesta cumple con el objetivo.

3. **Simulación**
   Esta es la parte en la que se ejecuta el código implementado y se aplican los conceptos desarrollados para comparar la nueva geometría con la actual y comprobar si se ha alcanzado el objetivo principal.

1.3. Estructura del trabajo

Capítulo 1 **Introducción 1**
Presentación del problema a resolver y la motivación de la propuesta de mejora presentada. Destaca la lista de propiedades del elemento principal, la curva de Hilbert, para compararla con otras geometrías fractales 1.1.1.

Capítulo 2 **Contexto y estado del arte 2**
En este capítulo se realiza una comparación del trabajo con diversas investigaciones y artículos previos. Asimismo, se establecen relaciones entre distintos estudios en función de los métodos empleados, la geometría propuesta y el enfoque adoptado para abordar el problema.

Capítulo 3 **Identificación de requisitos 3**
En este capítulo se describe la metodología empleada para la realización del estudio, así como las herramientas de software necesarias para la implementación de las simulaciones.

Capítulo 4 **Objetivos 4**
En este capítulo se define el objetivo principal del trabajo, así como los subobjetivos que guían el desarrollo de la investigación con el fin de alcanzar los resultados planteados.

Capítulo 5 **Curva que rellena el espacio 5**
En este capítulo se presenta la base teórica necesaria para el desarrollo y comprensión de la geometría del problema, la curva de Hilbert.

Capítulo 6 **Transferencia de calor 6**
En este capítulo se expone la base teórica necesaria para comprender los mecanismos

de intercambio de calor entre los distintos elementos, así como la evolución de su temperatura hasta alcanzar el equilibrio térmico.

Capítulo 7 **La curva de Hilbert 7**

En este capítulo se desarrollan los conceptos teóricos necesarios para la generación de figuras y se formalizan definiciones propias. Asimismo, se presenta el algoritmo mediante el cual se construye la geometría principal del problema y se incluyen ejemplos de geometrías generadas mediante código propio 7.13.

Capítulo 8 **Transferencia térmica sobre la curva de Hilbert 8**

En este capítulo se describe la aplicación de la transferencia térmica sobre la geometría desarrollada. En particular, se destaca la definición de la recurrencia 17, la cual modela la evolución de la temperatura en cada punto de la curva hasta alcanzar el equilibrio térmico.

Capítulo 9 **Codificación 9**

En este capítulo se presenta la mayor parte del código necesario para la ejecución de las simulaciones y la obtención de los resultados del estudio. Por esta razón, se encuentra vinculado de forma constante con el apéndice A.1, donde se incluyen las celdas de Python desarrolladas. Finalmente, se describe el funcionamiento de la función principal, en la que se integran los distintos elementos desarrollados a lo largo del trabajo.

Capítulo 10 **Resultados 10**

En este capítulo se aplican los conceptos desarrollados a lo largo del trabajo y se establece una comparación entre la geometría principal propuesta y la geometría convencional. Asimismo, se analiza el cumplimiento del objetivo principal del trabajo y se destacan las simulaciones, accesibles mediante un enlace, con el fin de mostrar el funcionamiento del algoritmo principal al detalle.

Capítulo 11 **Conclusiones y trabajo futuro 11**

En este capítulo final se evalúa el grado de cumplimiento de los subobjetivos planteados con el fin de validar el objetivo principal del estudio. Asimismo, se presentan las conclusiones generales del trabajo y se plantea una perspectiva de futuro orientada a posibles mejoras y a la continuidad de la investigación para optimizar su aplicación práctica.

2. Contexto y Estado del Arte

2.1. Mejorar la transferencia térmica a través de micro-geometria

En la siguiente investigación reciente, (Liu, Sun, Yin, Wang, y Sun, 2025), se encuentra este artículo sobre la mejora de transferencia térmica que busca aportar modificaciones geométricas de la tubería y añadir formas internas para intensificar la mezcla del flujo.

(Liu y cols., 2025) estudian un intercambiador basado en un tubo con hendiduras y proponen añadirle una cinta helicoidal en el interior para inducir flujo secundario y mejorar la utilización de las cavidades.
En los resultados se muestran incrementos del coeficiente medio de transferencia de calor frente al tubo convencional, destacando el compromiso entre mejora térmica pero penalización por pérdida de carga.

La diferencia principal frente a esta investigación es que se aborda el problema desde una perspectiva geométrica general: explorar si una curva plana que "rellena el espacio" (en particular, la curva de Hilbert) puede mejorar también la transferencia térmica.

Comparando ambos enfoques, (Liu y cols., 2025) priorizan la intensificación local del intercambio mediante elementos internos y métricas termo-hidráulicas. En cambio, este trabajo prioriza el trazado global del conducto como variable principal del estudio, donde la eficiencia puede emerger de cómo cubrir el espacio y cómo se reparte el gradiente térmico.
En definitiva, en cada uno de estos estudios se estudia el sistema de transferencia térmica de forma distinta. (Liu y cols., 2025) trabajan con mejoras de la micro-geométrica (rugosidades/hendiduras) y esta investigación trabaja con la macro-geométrica (topología del recorrido).

Tabla 2.1: Comparación entre estudios sobre la mejora de transferencia térmica

Criterio	**(Liu y cols., 2025)**	**(TFM, 2026)**
Problema abordado	Mejora de la transferencia de calor mediante intensificación termo-hidráulica	Mejora de la transferencia térmica mediante el diseño geométrico del recorrido
Enfoque principal	Optimización local del intercambio térmico mediante elementos internos (dimples y cinta helicoidal)	Optimización global del intercambio térmico mediante la trayectoria geométrica
Geometría analizada	Tubo con hendiduras y cinta helicoidal	Curva de Hilbert
Metodología	Simulación numérica *Computational Fluid Dynamics* (**CFD**)	Simulación numérica Python
Escala de optimización	Micro-geométrica (interacción local fluido-superficie)	Macro-geométrica (topología global del recorrido)

2.2. Intensificar la transferencia térmica con hendiduras

En el artículo escrito por (Jaffal, Mahmoud, Imran, y Hasan, 2023), los autores también buscan intensificar la transferencia térmica mediante la introducción de hendiduras en los canales de refrigeración.
(Jaffal y cols., 2023) analizan un canal de forma serpentina con hendiduras inclinadas aplicado a la refrigeración líquida de módulos de baterías de litio.

En el estudio se propone una placa refrigerada por canal serpentín con hendiduras internas orientadas en distintos ángulos y con diferentes geometrías (semicircular, trapezoidal y triangular). Análogamente al artículo anterior nombrado, el objetivo principal es inducir flujos secundarios que mejoran la mezcla del fluido aumentando el coeficiente de transferencia de calor.
Este trabajo combina simulaciones CFD tridimensionales con validación experimental, analizando el comportamiento termo-hidráulico como en la anterior publicación.

Los resultados muestran incrementos significativos mucho más altos que el anterior

trabajo; no obstante, estas mejoras térmicas van acompañadas de un aumento de la pérdida de carga también mayor, que era una de las premisas en el anterior estudio a no perjudicar.

La principal diferencia con respecto a este TFM está también en el enfoque geométrico adoptado. Mientras que (Jaffal y cols., 2023) intensifican la transferencia térmica mediante modificaciones locales de la micro-geométrica interna del canal, este estudio aborda el problema desde una perspectiva geométrica global.

En definitiva, como en la anterior comparación, ambos estudios persiguen el mismo objetivo, –mejorar la transferencia térmica– pero desde escalas geométricas diferentes.

Tabla 2.2: Comparación entre estudios sobre intensificación de transferencia térmica

Criterio	**(Jaffal y cols., 2023)**	**(TFM, 2026)**
Sistema estudiado	Placa refrigerada con canal serpentín para baterías de litio	Recorrido del flujo en superficie bidimensional para transferencia térmica
Estrategia de mejora	Introducción de hendiduras inclinadas con diferentes geometrías	Diseño de una trayectoria geométrica compleja que rellena el espacio
Tipo de geometría	Canal serpentín con hendiduras internas	Curva de Hilbert bidimensional
Metodología	Simulación CFD tridimensional y validación experimental	Modelado geométrico y simulación numérica
Indicadores de desempeño	Número de Nusselt y pérdida de carga	Distribución térmica, eficiencia del intercambio y equilibrio térmico
Escala de optimización	Micro-geométrica	Macro-geométrica

2.3. Modificaciones estructurales en transferencia térmica

La investigación de (Yuan, Cao, Zhang, Xiao, y Wang, 2024) aborda la mejora de la transferencia térmica centrándose en la optimización termo-hidráulica de intercambiadores de calor de tubos helicoidales multicapa. Busca aportar modificaciones estructurales al diseño del equipo para guiar el flujo en el lado de la carcasa y, con ello, intensificar el

intercambio térmico.

(Yuan y cols., 2024) estudian un intercambiador de calor de tubos helicoidales multicapa con doble paso por carcasa (*DSMHCTHE*, así es como lo identifican), al que incorporan cilindros guía concéntricos para redirigir el flujo del fluido frío alrededor de los tubos helicoidales. Esta modificación aumenta el contacto efectivo entre el fluido y la superficie de intercambio.

El trabajo muestra incrementos significativos del coeficiente global de transferencia de calor respecto al diseño convencional (*MHCTHE*). Sin embargo, como se ha visto en la anterior publicación, añadir elementos internos dentro de la tubería va acompañado de un aumento de la pérdida de carga.

La diferencia principal frente a esta investigación es que se aborda el problema desde una perspectiva geométrica del espacio y no desde la tubería en sí.
Comparando ambos enfoques, (Yuan y cols., 2024) priorizan la intensificación local del intercambio térmico mediante modificaciones estructurales internas. En cambio, este trabajo prioriza el trazado global del conducto como variable de diseño, donde la mejora en eficiencia térmica puede emerger de cómo se cubre el dominio y se reparte el gradiente térmico.

Tabla 2.3: Comparación entre estudios sobre mejoras de transferencia térmica estructurales

Criterio	**(Yuan y cols., 2024)**	**(TFM, 2026)**
Problema abordado	Mejora de la transferencia térmica mediante rediseño estructural del intercambiador	Mejora de la transferencia térmica mediante el diseño geométrico del recorrido
Enfoque principal	Optimización termo-hidráulica del flujo en el lado de la carcasa	Optimización global del intercambio térmico mediante la trayectoria
Geometría analizada	Tubos helicoidales multicapa con doble paso por carcasa	Curva de Hilbert
Metodología	Simulación numérica CFD y validación experimental	Simulación numérica en Python
Mecanismo de mejora	Redistribución del flujo y aumento del contacto fluido–superficie	Mejor cobertura espacial y distribución uniforme del calor
Escala de optimización	Estructura del recorrido	Topología del recorrido

2.4. Estudio aplicado sobre la geometría del recorrido

Dentro de los estudios orientados al diseño geométrico del recorrido del fluido, se puede encontrar el artículo de (Radzai, Yaw, Lim, Koh, y Ahmad, 2021), donde presentan un análisis numérico detallado del desempeño térmico en paneles de *refrigeración radiante* (**RCP**) con diferentes configuraciones de tubería basadas todas en geometrías serpentín. El objetivo principal del estudio es maximizar la capacidad de enfriamiento y mejorar la uniformidad de la distribución térmica sin incrementar excesivamente la pérdida de carga.

El trabajo evalúa, mediante simulaciones CFD en ANSYS Fluent, tres configuraciones de flujo: un serpentín convencional de una entrada y una salida, un serpentín contracorriente de dos entradas y dos salidas, y un diseño con tres entradas y tres salidas. Los resultados muestran que la configuración contracorriente presenta la mejor uniformidad térmica y la mayor capacidad de enfriamiento, además de una menor caída de presión respecto al serpentín convencional. Además, el estudio destaca la relación directa entre la

topología del recorrido, el número de codos y la eficiencia termo-hidráulica del sistema.

Por otra parte, este trabajo aborda el problema desde una perspectiva geométrica más abstracta, proponiendo el uso de la curva de Hilbert como trayectoria del flujo.
A diferencia del estudio de (Radzai y cols., 2021), el TFM no se centra en la optimización de configuraciones serpentín ya existentes, sino en la exploración conceptual de una nueva geometría que sea más eficiente.

La comparación entre estos estudios enseña dos maneras diferentes de abordar el mismo problema. Mientras (Radzai y cols., 2021) optimizan la geometría del serpentín ya implementada, el TFM explora una topología de recorrido nueva y no convencional que podría servir como base para el desarrollo de nuevas configuraciones.

Tabla 2.4: Comparación sobre diseños geométricos del recorrido

Criterio	**(Radzai y cols., 2021)**	**(TFM, 2026)**
Problema abordado	Mejora de la transferencia térmica mediante geometría del canal de serpentín	Mejora de la transferencia térmica mediante geometría del canal no convencional
Sistema estudiado	Panel de refrigeración radiante con tuberías embebidas	Recorrido de flujo bidimensional para transferencia térmica
Objetivo principal	Optimizar la capacidad de enfriamiento y uniformidad térmica	Evaluar trayectorias geométricas para mejorar la distribución térmica
Geometría del recorrido	Serpentín convencional y serpentines multientrada	Curva de Hilbert
Estrategia de mejora	Modificación del recorrido del serpentín	Optimización del trazado del recorrido
Metodología	Simulación CFD tridimensional en ANSYS Fluent	Modelado geométrico y simulación numérica con Python
Indicadores analizados	Capacidad de enfriamiento, uniformidad térmica y pérdida de carga	Distribución y uniformidad térmica
Limitaciones	Limitado a configuraciones serpentín y condiciones estacionarias	Falta de validación experimental directa

2.5. Conclusión

A partir de los anteriores artículos, se pueden extraer varias conclusiones relevantes para fundamentar este trabajo.

En primer lugar, el estudio de (Liu y cols., 2025) confirma que una buena herramienta de simulación es esencial para este tipo de trabajos e investigaciones. Los modelos CFD permiten no solo cuantificar el coeficiente de transferencia de calor, sino también observar fenómenos físicos como la mezcla del flujo, las turbulencias, la interacción fluido-superficie, etc.

En segundo lugar, el trabajo de (Jaffal y cols., 2023) aporta una perspectiva detallada sobre cómo estudiar la transferencia térmica en canales y tuberías. Su análisis evidencia que la introducción de hendiduras permite mejorar significativamente el intercambio térmico, pero se ve por primera vez el grave problema de pérdida de carga.

Sin embargo, los resultados más significativos para el planteamiento de esta investigación provienen del trabajo de (Radzai y cols., 2021). Sus simulaciones sobre paneles de refrigeración radiante con diferentes configuraciones serpentín confirman que la geometría del recorrido es un factor determinante en la uniformidad térmica y la eficiencia del sistema. No obstante, los diseños analizados se limitan a variaciones del serpentín que ya se conoce, lo que deja abierta la posibilidad de explorar trayectorias nuevas como la de este trabajo y ver si se puede conseguir una mejor distribución.

En definitiva, esta investigación propone crear nuevas situaciones para mejorar el resultado y no buscar el mismo objetivo modificando la estructura ya conocida y tradicional. Esta diferencia de enfoque sugiere que las curvas fractales podrían integrarse como una evolución de los diseños convencionales, combinando una mejor cobertura espacial con una distribución más uniforme del calor y sin necesidad de incrementar la pérdida de carga.

En conjunto, el estado del arte evidencia que las mejoras en transferencia térmica pueden abordarse tanto desde la micro-geometría como desde la macro-geometría. Sin embargo, existe una clara brecha de investigación en la integración de trayectorias geométricas complejas y nuevas dentro del diseño de sistemas térmicos, ya que no se ha encontrado ninguna investigación que implemente nuevas tuberías para este campo.
Esto justifica plenamente la originalidad y relevancia de este estudio y trabajo realizado.

Tabla 2.5: Resumen comparativo de enfoques para la mejora de la transferencia térmica

Criterio	**(Liu y cols., 2025)**	**(Jaffal y cols., 2023)**	**(Yuan y cols., 2024)**	**(Radzai y cols., 2021)**	**(TFM, 2026)**
Uso de simulación numérica	✓	✓	✓	✓	✓
Uso de simulación CFD	✓	✓	✓	✓	×
Validación experimental	×	✓	✓	×	×
Mejora por micro-geometría	✓	✓	×	×	×
Mejora por macro-geometría	×	×	×	✓	✓
Optimización del recorrido	×	×	✓	✓	✓
Análisis de pérdida de carga	✓	✓	✓	✓	×
Enfoque industrial aplicado	✓	✓	✓	✓	✓
Enfoque innovador	×	×	×	×	✓
Uso de geometría no convencional	×	×	×	×	✓
Búsqueda de uniformidad térmica	✓	✓	✓	✓	✓

3. Identificación de requisitos

3.1. Metodología

La presente investigación se organiza con un **enfoque hipotético-inductivo**. Este enfoque parte de una hipótesis general que da estructura al objetivo principal del trabajo. A partir de esta idea, se crea de forma lógica y ordenada un listado de pasos que formalizan el desarrollo del estudio y sirven para definir los objetivos específicos.

Asimismo, se siguen las tres fases fundamentales de todo proyecto de investigación:

1. **Propuesta:** Incluye los primeros capítulos, donde se introduce el problema, su contexto y relevancia en el día a día, así como los objetivos planteados.

2. **Anteproyecto:** Fase en la que se define el marco teórico del trabajo y se explican todas las variables que se utilizarán en el estudio.

3. **Proyecto:** Capítulos principales del TFM en los que se desarrollan los conceptos relativos al objeto de estudio y se realizan las pruebas necesarias para comprobar y demostrar el objetivo principal.

En cuanto a la metodología adoptada, el trabajo se enmarca en una **metodología mixta** y un **diseño concurrente**. Este enfoque implica la integración simultánea de análisis y resultados de forma cuantitativa y cualitativa. De este modo, se obtiene una visión más holística del objeto de estudio y se analiza de forma mas exhaustiva la problemática abordada.

3.2. Software

Para llevar a cabo el análisis y los resultados, se desarrollan e implementan todas las funciones en **Python** sobre un entorno **Jupyter Notebook**. Este lenguaje dispone de múltiples librerías que permiten simular con precisión el objeto de estudio. La herramienta Notebook facilita la separación de cada parte de la simulación, permitiendo estructurar y organizar con claridad la parte práctica.

A partir de los diferentes resultados obtenidos en las distintas simulaciones, se contrastan los comportamientos de los modelos para validar las hipótesis planteadas.

4. Objetivos

4.1. Objetivo principal

Analizar y comparar, mediante simulaciones, la transferencia térmica de la geometría de un canal basado en la curva de Hilbert frente al serpentín convencional sobre una superficie.

4.2. Objetivos específicos

- Definir formalmente una curva que rellene el espacio.
- Desarrollar la teoría sobre transferencia térmica y establecer las leyes que rigen el intercambio de calor del fluido que circula por el canal.
- Codificar, a partir de la definición formal, una curva que rellene el espacio de un rectángulo cualquiera.
- Desarrollar la recurrencia general de la temperatura del fluido que circula por el canal hasta alcanzar el equilibrio térmico.
- Simular la recurrencia de temperatura mediante cálculo computacional hasta llegar al equilibrio térmico.
- Analizar la situación de equilibrio térmico y compararla con la trayectoria del serpentín convencional.

5. Curva que rellena el espacio

En este trabajo se estudiarán diferentes elementos, los cuales no todos estarán en la misma dimensión; es decir, habrán objetos que estarán definidos en el espacio 1-dimensional $\mathbb{R}$, otros en el 2-dimensional $\mathbb{R}^2$ y características que dependerán del 3-dimensional $\mathbb{R}^3$. Pero para simplificar definiciones y notaciones, se trabajará sobre subconjuntos de estos sin perder generalidad, los cuales se denotan como:

$\mathcal{I} = [0,1] \subset \mathbb{R}$

$\mathcal{Q} = [0,1]^2 \subset \mathbb{R}^2$

$\mathcal{W} = [0,1]^3 \subset \mathbb{R}^3$

Definición 1. *Se define $\mathcal{J}_n$ como la* ***medida de Jordan n-dimensional****; esta viene descrita por:*

$$\mathcal{J}_n(A) = \int_R \mathcal{X}_A$$

donde A es un subconjunto de $\mathbb{R}^n$, R es un rectángulo que contiene el conjunto A y $\mathcal{X}_A$ es la función característica de A.

Si A es un subconjunto de $\mathbb{R}^n$, la función característica de A se define como:

$$\mathcal{X}_A : \mathbb{R}^n \longrightarrow \mathbb{R} \quad \text{tal que} \quad \mathcal{X}_A(x) = \begin{cases} 1 & \text{si} \quad x \in A \\ 0 & \text{si} \quad x \notin A \end{cases}$$

Por tanto, se puede afirmar que $\mathcal{J}_n(A)$ mide el área o volumen del conjunto.

Como en este estudio se simulan curvas sobre un espacio finito y real, se puede afirmar que todo conjunto será un espacio euclidiano y, por tanto, se puede sustitur $\mathbb{R}^n$ por $\mathbb{E}^n$, ya que el espacio donde estarán definidos todos los elementos será finito y dotado de distancia (producto escalar).

Definición 2. *Se define $f_*(A)$ como la imagen directa de A mediante la función f. Esta puede representarse de la siguiente forma:*

$$f_*(A) = \{f(x) \in \mathcal{R}(f) \quad | \quad x \in A \cap \mathcal{D}(f)\}$$

Donde f es una función entre espacios euclidianos, $\mathcal{R}(f)$ indica el rango o imagen de la función y $\mathcal{D}(f)$ el dominio de esta.

Ahora ya se puede definir lo que es una curva que rellena el espacio.

Definición 3. *Se puede* ***parametrizar una curva*** *a partir de la función α descrita de la siguiente manera:*

$$\alpha : \mathcal{I} \longrightarrow \mathbb{E}^n$$

Una aplicación infinitamente diferenciable y continua.

Entonces se denota ***curva*** *a $\alpha_*(\mathcal{I})$, es decir, la imagen directa de $\mathcal{I}$ mediante la función α, en la que se puede identificar el punto inicial como $\alpha(0)$ y el punto final como $\alpha(1)$.*

Se representa la curva como $\mathcal{C}$, es decir, $\mathcal{C} = \alpha_*(\mathcal{I})$.

- Se dice que una curva es simple si no se corta en ningún punto consigo misma, es decir, $\alpha(t_1) \neq \alpha(t_2) \quad \forall t_1, t_2 \in \mathcal{I}$.

Definición 4. *Dada la curva $\alpha : \mathcal{I} \longrightarrow \mathbb{E}^n$ con $n \geqslant 2$, si esta cumple que $\mathcal{J}_n(\alpha_*(\mathcal{I})) > 0$, entonces $\mathcal{C} = \alpha_*(\mathcal{I})$ se llama* ***curva que rellena el espacio****.*

Proposición 1. ***Cantor****. Existen aplicaciones $f_h : \mathcal{I} \longrightarrow \mathcal{Q}$ bien definidas, continuas, exhaustivas y no diferenciables en ningún punto.*

Demostración. Si t es el punto final de uno de los subintervalos de $\mathcal{I}$, entonces t pertenece a dos secuencias diferentes de intervalos anidados, pero como intervalos adyacentes se asignan a cuadrados de $\mathcal{Q}$ adyacentes, esto conduce a la misma imagen; es decir, f_h está bien definida.

Es exhaustiva porque, dado cualquier punto $p \in \mathcal{Q}$, se puede dar una sucesión de subcuadrados encajados cuya intersección es p, y esta subsucesión de subcuadrados tiene asociada una sucesión de subintervalos encajados en $\mathcal{I}$ que define un cierto punto $t \in \mathcal{I}$ que obviamente cumple $f_h(t) = p$.

La aplicación f_h es continua, pues si $|t - t'| < \frac{1}{4^n}$ entonces o bien pertenece al mismo intervalo $\mathcal{I}$ de la etapa n o bien pertenece a dos subintervalos consecutivos y, por ello, $f_h(t)$ y $f_h(t')$ o bien pertenecen al mismo subcuadrado o bien pertenecen a subcuadrados adyacentes, de donde $||f_h(t) - f_h(t')|| \leqslant \frac{\sqrt{5}}{2^n}$. Esto prueba la continuidad uniforme de f_h en $\mathcal{I}$.

Falta probar que la función f_h no es derivable en ningún punto del intervalo $\mathcal{I}$; para ello, ninguna de sus dos funciones componentes lo son. Pongamos $f_h = (\varphi, \psi)$.

Se fija $t \in \mathcal{I}$; sea $n \geqslant 3$ para t_n de modo que $|t - t_n| \leqslant \frac{10}{2^{2n}}$ y que $f_h(t)$ esté separado de $f_h(t_n)$ horizontalmente por al menos un subcuadrado, esto es,

$$|\varphi(t) - \varphi(t_n)| \geqslant \frac{1}{2^n}.$$

Esto siempre es posible gracias a cómo se ordenan los subcuadrados de un mismo orden.

Entonces se tiene

$$\frac{|\varphi(t) - \varphi(t_n)|}{|t - t_n|} \geqslant \frac{\frac{1}{2^n}}{\frac{10}{2^{2n}}} = \frac{2^n}{10},$$

con lo que se demuestra que φ no puede ser derivable en t. Lo mismo puede hacerse con ψ y, por tanto, f_h no es derivable en ningún punto. (Sagan, 1994) $\square$

Esto da una demostración de que existen curvas que llenan el espacio euclidiano 2-dimensional y están bien definidas. Un ejemplo es la estructura principal de este trabajo: la curva de Hilbert.

6. Transferencia de calor

La **transferencia térmica** o **transferencia de calor** es la ciencia que determina las razones por las que se transfiere la energía de un sistema a otro como resultado de la diferencia de temperatura.
Esta energía se llama **calor**, y es una de las muchas formas en las que se puede expresar la energía. Con base en la experiencia del día a día, se sabe que estos tipos de intercambios de energía siempre van del medio caliente al medio frío, es decir, del medio con una temperatura más alta hacia el de temperatura más baja, y dicha transferencia se detiene al alcanzar ambos medios la misma temperatura, el **equilibrio térmico**.

Sin diferencia de temperatura no hay transferencia térmica.

Actualmente el término calor, que es cantidad de energía, está arraigado de forma equivocada en algunos contextos, aunque se sobreentienda su significado.
Por ejemplo, al hablar de *flujo de calor*, no se refiere uno al flujo de un fluido que se llama calor, sino a la transferencia de energía térmica.
Por eso, ya se advierte que en este trabajo se hace, como en el día a día del ámbito científico, un abuso del lenguaje para que sea más amena la lectura y con menos tecnicismos.
Se llamará *calor* a la energía térmica y *transferencia de calor* al intercambio de energía térmica.
En cuanto a la notación, se aplicará la misma que por convenio: Se denota **Q**, la cantidad total de calor transferido en un proceso.

Estos cambios de energía pueden determinarse de muchas formas distintas dependiendo de las situaciones en las que estén sucediendo, ya que según qué propiedades se cumplan se definirán de una forma u otra. Para este caso, se estudia la transferencia térmica de un fluido que circula por una tubería, es decir, un sistema en el que no cambia la forma ni el estado de los elementos y, por tanto, se puede asegurar que se trata de un **proceso estacionario**. Estos tipos de procesos son aquellos en los que el intercambio de calor total entre el sistema es completo y no se pierde energía en el proceso.
Además, este problema se puede definir como un *estudio de balances de energía en sistemas de flujo estacionario*, ya que se trata de una situación estacionaria donde la cantidad de masa que circula a través de la sección transversal del dispositivo de flujo, la curva de Hilbert, es también constante.
Y como en el intercambio de energía no hay trabajo, sino solo flujo de calor, y se consideran

negligibles las variaciones de velocidad del fluido y su energía potencial, el balance de flujo estacionario se reduce a la **fórmula del calor sensible**:

$$\mathbf{Q} = m\ c_p\ \Delta T \tag{6.1}$$

Donde c_p es el calor específico a presión constante, m la masa del fluido y ΔT la diferencia de temperatura.

Pero para el estudio de la problemática abordada, se analizará la energía de forma diferente: Se estudiará el calor en un punto de la tubería como la suma de los calores proporcionados por los tres mecanismos distintos en que se produce la transferencia térmica: la **conducción**, la **convección** y la **radiación**.

$$\mathbf{Q} = Q_{\text{cond.}} + Q_{\text{conv.}} + Q_{\text{rad.}} \tag{6.2}$$

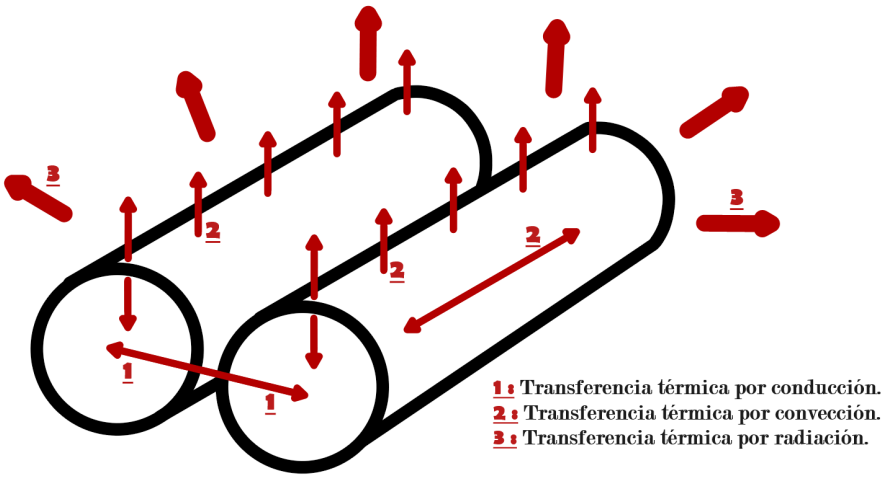


Figura 6.1: Transferencia térmica del fluido por una tubería. (Rodrigo Arbós. 2026)

6.1. Conducción

Definición 5. *Se llama **conducción** al tipo de transferencia de energía de las partículas más energéticas de una sustancia hacia las adyacentes menos energéticas como resultado de interacciones entre las partículas.*

Este proceso depende de la geometría del medio que separa las partículas que intercambian energía, su espesor, el material que las separa y, obviamente, la diferencia de temperatura que hay entre ellas.

Experimentalmente se ha podido concluir la siguiente relación:

Definición 6. *La conducción de calor a través de una capa plana es proporcional a la diferencia de temperatura a través de esta y al área de transferencia de calor, pero inversamente proporcional al espesor de la capa.*

$$\mathbf{Q_{cond.}} = -k\ A\ \frac{\Delta T}{\Delta X}$$

Donde $k :=$ Conductividad térmica del material.
$A :=$ Área sobre la que sucede la conducción.
$\Delta X :=$ Espesor del material o distancia entre partículas que participan en la conducción.
$\Delta T :=$ Diferencia de temperatura entre las partículas de la transferencia térmica.

Definición 7. *La **conductividad térmica** de un material es la medida de la capacidad que tiene un material para conducir el calor a través de él.*

El valor negativo de la fórmula asegura que $\mathbf{Q_{cond.}}$ sea positiva, ya que, como se ha explicado, el gradiente de la temperatura siempre será negativo en las transferencias de energía, pues el calor fluye del medio caliente al medio frío y, por tanto, $\Delta T < 0$.

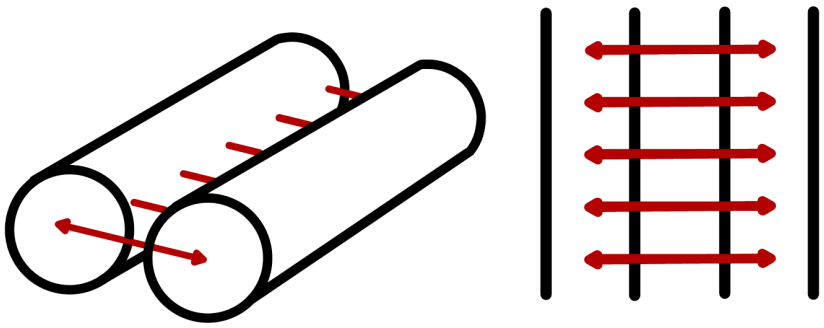


Figura 6.2: Transferencia térmica por conducción entre tuberías. (Rodrigo Arbós. 2026)

En este estudio se puede agrupar el término espesor del material con la constante de conductividad y así despreciar la variable ΔX. Para este problema, la transferencia por conducción siempre va a ser entre las paredes de la misma tubería y separadas a la misma distancia en todo punto; por tanto, el espesor entre puntos de conducción puede considerarse una constante más y facilitar la ecuación de la siguiente manera:

$$\mathbf{Q_{cond.}} = -k\ A\ \Delta T \tag{6.3}$$

6.2. Convección

Definición 8. *La **convección** es el modo de transferencia de energía entre una superficie sólida y el líquido o gas adyacente que está en movimiento y comprende los efectos combinados de la conducción y el movimiento de fluidos.*

A medida que aumenta la velocidad del flujo, se incrementa el coeficiente de transferencia de calor. Esto es porque el intercambio energético entre el fluido y la superficie ocurre con mayor frecuencia.

En ausencia de movimiento del fluido no existiría transferencia por convección, de modo que el único mecanismo de transferencia térmica disponible sería la conducción.
Al estar un fluido en movimiento se obtiene mayor transferencia térmica, pero es más compleja a la hora de determinar la cantidad total.

En resumen, la convección facilita una mayor transferencia térmica y es la razón por la cual se tiene el fluido en movimiento. Con estas transferencias de energía aumenta mucho más el número de intercambios.

Definición 9. *Se denomina **transporte de masa** al suceso por el cual el fluido avanza por el canal.*

Como ya se ha explicado, sobre un punto del fluido habrá transferencia térmica por conducción y este, al estar sometido al transporte de masa, se desplazará y creará otros intercambios de energía con distintas temperaturas, mejorando y acelerando así la transferencia térmica.
Es decir, al estar el fluido en movimiento aparecen los procesos de convección que, sumados a los de conducción y radiación ya existentes, permiten alcanzar el equilibrio térmico con más rapidez.

Se pueden distinguir dos tipos de convección:

- **Convección forzada:** Sucede si el fluido es forzado a moverse sobre la superficie mediante un medio externo como ventilador, bomba, viento, etc.
- **Convección natural:** Sucede si el fluido está en movimiento por fuerzas de empuje inducidas por diferencias de densidad debido a la variación de temperatura en el fluido.

A pesar de toda la complejidad de la convección, se ha observado que la razón que la define siempre es la misma:

Definición 10. *La rapidez de la transferencia de calor por convección es proporcional a la diferencia de temperatura y se expresa en forma conveniente por* ***la ley de Newton del enfriamiento****.*

$$\mathbf{Q_{conv.}} = h\ A\ \Delta T \tag{6.4}$$

Donde $h :=$ Coeficiente de transferencia de calor por convección.
$A :=$ Área superficial a través de la cual tiene lugar la transferencia de calor.
$\Delta T :=$ Diferencia de temperatura entre los puntos del fluido. Se debe remarcar que h es un parámetro que se determina de forma experimental y es la variable que diferencia las distintas convecciones que pueden existir en un mismo proceso.

En esta situación estudiada, se observan dos intercambios distintos por convección:

1. **Convección interna:** Surge de la interacción del fluido consigo mismo por estar en movimiento. Aparece al tener una diferencia de temperatura en dos puntos consecutivos.

2. **Convección externa:** Surge del intercambio de energía entre el fluido que está en movimiento por el canal y el cuerpo que envuelve la tubería.

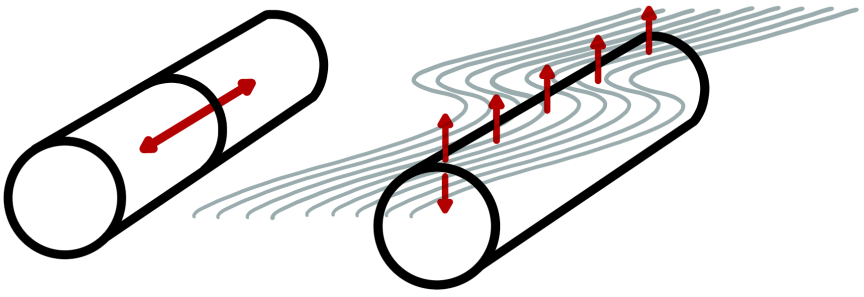


Figura 6.3: Transferencia térmica por convección por una tubería. (Rodrigo Arbós. 2026)

6.3. Radiación

Definición 11. *La* ***radiación*** *es la energía emitida por la materia en forma de ondas electromagnéticas (o fotones) como resultado de los cambios en las configuraciones electrónicas de los átomos o moléculas.*

A diferencia de la conducción y la convección, esta transferencia de calor no requiere de un medio intermedio; de hecho, es la más rápida y no sufre atenuación.

Hay muchos tipos de radiación; en este estudio claramente se trabaja con la radiación térmica, que es la forma de radiación emitida por los cuerpos debido a su temperatura. Otras como la radiación de rayos X, gamma, de microondas, de radio y televisión, en cambio, no están relacionadas con la temperatura.

Está demostrado que todo cuerpo con temperatura por encima del cero absoluto emite radiación térmica, y que la radiación es un *fenómeno volumétrico*, es decir, todo sólido, líquido o gas emite, absorbe o transmite radiación.

Definición 12. *La razón máxima de la radiación que se puede emitir desde una superficie con una temperatura termodinámica T_s es expresada por la* ***ley de Stefan-Boltzmann*** *como:*

$$\mathbf{Q}_{emitida,\ máx.} = \sigma\ A\ {T_s}^4$$

donde $\sigma = 5,670 \cdot 10^{-8}\ \frac{W}{m^2K^4}$ es la constante de *Stefan-Boltzmann.*

Pero esto solo sucede con la superficie del **cuerpo negro**, una superficie idealizada ya que se considera que emite radiación de forma máxima. En realidad, toda superficie a la misma temperatura que esta emite calor por radiación con menor intensidad. Esto se expresa como:

$$\mathbf{Q}_{\text{emitida}} = \epsilon\ \sigma\ A\ {T_s}^4$$

donde ϵ es la **emisividad de la superficie**, propiedad cuyo valor está en el intervalo $(0, 1)$, es decir, $0 \leqslant \epsilon \leqslant 1$. La *emisividad* mide cuán cerca está una superficie de comportarse como un cuerpo negro.

De la misma forma que una superficie emite energía, también la absorbe. Otra propiedad de la radiación de una superficie es la **absortividad** α, que actúa de manera análoga a la emisividad.
El valor de α también pertenece al intervalo $(0, 1)$, $0 \leqslant \alpha \leqslant 1$, y de la misma manera que con ϵ, si $\alpha = 1$, la superficie es un cuerpo negro y absorbería toda la radiación incidente.

$$Q_{\text{absorbida}} = \alpha \cdot Q_{\text{incidente}}$$

En general, ϵ y α de una superficie dependen de la temperatura y la longitud de onda. La **ley de Kirchhoff** de la radiación establece que la emisividad y la absortividad de una superficie a una misma temperatura y longitud de onda son iguales.

Si se juntan todos estos conceptos, se puede definir la diferencia entre la radiación emitida y la radiación absorbida como la *transferencia neta de calor por radiación*. Se dice que una *superficie gana energía por radiación* si la absorción es mayor que la emisión; de manera opuesta, se dice que *pierde energía por radiación*.

En base a esta definición, se puede formalizar el proceso de transferencia térmica por radiación de la siguiente manera:

Definición 13. *Una superficie de emisividad ϵ y área superficial A_s a una temperatura termodinámica T_{int} encerrada completamente por otra superficie más grande a temperatura termodinámica T_{ext} y separada por un gas que no interfiere, la razón neta de la transferencia de calor por radiación sobre la superficie viene dada por:*

$$\mathbf{Q_{rad}} = \epsilon \ \sigma \ A \ \left({T_{int}}^4 - {T_{ext}}^4\right)$$

En la que, si se agrupan las constantes, se puede representar de la siguiente forma:

$$\mathbf{Q_{rad}} = q \ A \ \left({T_{int}}^4 - {T_{ext}}^4\right) \tag{6.5}$$

Donde:

$q :=$ Constante térmica que participa en el proceso de radiación.

$A_s :=$ Área por donde el fluido emite la radiación.

$T_{int} :=$ Temperatura del fluido.

$T_{ext} :=$ Temperatura del exterior que absorbe la radiación emitida por el fluido.

Se señala que la transferencia de calor por radiación hacia una superficie rodeada por un gas como el aire, ocurre de forma paralela a la convección entre esa superficie y el gas.

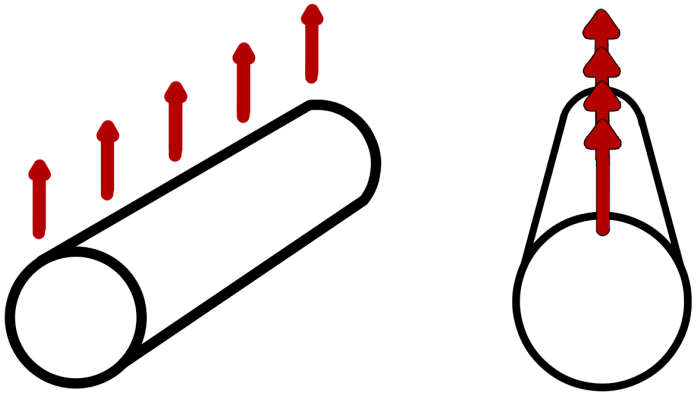


Figura 6.4: Transferencia térmica por radiación entre superficies. (Rodrigo Arbós. 2026)

(Çengel y Ghajar, 2020)

7. La Curva de Hilbert

Como se ha introducido anteriormente, la curva de Hilbert es una de las curvas que rellenan el espacio. Esta, por lo que se sabe, se describió en 1891 por el matemático David Hilbert en su articulo publicado en (Hilbert, 1891).

Su historia comienza a finales del siglo XIX, dentro del contexto del estudio de la continuidad y la dimensionalidad en análisis matemático. En 1890, el matemático italiano Giuseppe Peano en (Peano, 1890), presentó la primera curva continua que llenaba completamente un cuadrado, un resultado que sorprendió mucho a la comunidad matemática de esa época, ya que contradecía la intuición sobre las curvas unidimensionales.
Inspirado por este descubrimiento, David Hilbert publicó en 1891 (Hilbert, 1891), allí se encuentra el trabajo que hoy se utiliza para este proyecto. Creó una construcción alternativa, más geométrica e intuitiva, que hoy se conoce como la curva de Hilbert. Su versión era mucho más fácil de visualizar y analizar, lo que la convirtió en una herramienta importante en el estudio de la topología y los fundamentos de la geometría. (Mandelbrot, 1982)

Hilbert la definió en su trabajo como una curva fractal continua que recubre todo el plano donde está acotada, en la que cada una de las curvas que aproximan a la curva final es simple y su definición sigue un proceso recursivo.

7.1. Definición formal

1. Se divide el intervalo unidad $\mathcal{I}$ en cuatro subintervalos iguales,

$$\mathcal{I}_1 = [0, \frac{1}{4}], \quad \mathcal{I}_2 = [\frac{1}{4}, \frac{1}{2}], \quad \mathcal{I}_3 = [\frac{1}{2}, \frac{3}{4}], \quad \mathcal{I}_4 = [\frac{3}{4}, 1]$$

   y el cuadrado unidad $\mathcal{Q}$ en cuatro subcuadrados $\mathcal{Q}_1, \mathcal{Q}_2, \mathcal{Q}_3, \mathcal{Q}_4$ cada uno con longitud lateral $1/2$ y de centro $(1/4, 1/4)$, $(1/4, 3/4)$, $(3/4, 3/4)$ y $(3/4, 1/4)$ respectivamente. Luego se asigna cada subintervalo a uno de los subcuadrados de la siguiente forma:

   - El intervalo $\mathcal{I}_1$ se asigna al subcuadrado inferior izquierdo $\mathcal{Q}_1$.
   - El intervalo $\mathcal{I}_2$ al subcuadrado superior izquierdo $\mathcal{Q}_2$.
   - El intervalo $\mathcal{I}_3$ al subcuadrado superior derecho $\mathcal{Q}_3$.
   - Finalmente, el intervalo $\mathcal{I}_4$ al subcuadrado inferior derecho $\mathcal{Q}_4$.

   (7.1 (b))

2. Posteriormente, se hace lo mismo pero con cada subintervalo y los nuevos subcuadrados. Es decir, se divide cada subintervalo de $\mathcal{I}$ del paso anterior en cuatro subintervalos nuevos de igual longitud, para la segunda iteración un valor de 1/16. Estos de nominaran como $\mathcal{I}_{kj}$ con $k, j = 1, 2, 3, 4$.

$$\mathcal{I}_{kj} = [\frac{k-1}{4} + \frac{j-1}{16}, \frac{k-1}{4} + \frac{j}{16}]$$

y se divide de la misma forma que antes cada subcuadrado $\mathcal{Q}_k$ en cuatro subcuadrados $\mathcal{Q}_{kj}$, cada uno con una longitud lateral de esta vez 1/4.

Sin embargo, la correspondencia entre ellos no es cualquiera: se crea la biyección $\mathcal{I}_{kj} \leftrightarrow \mathcal{Q}_{kj}$ de modo que los subintervalos consecutivos están en biyección con subcuadrados adyacentes, es decir, tienen que compartir un lado.

(7.1 (c))

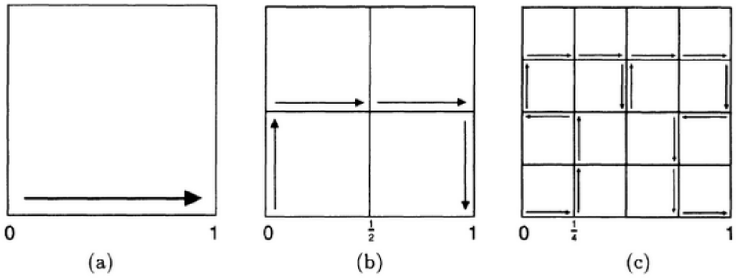


Figura 7.1: Mapeando el cuadrado. (Sagan, 1994)

(Sagan, 1994)

7.2. Propiedades

Definición 14. *Todo $t \in \mathcal{I}$ está determinado de forma única por una secuencia de intervalos cerrados anidados generados mediante particiones sucesivas. Las longitudes de estos intervalos tienden a* 0.

Esta secuencia de intervalos tiene en biyección una única secuencia de cuadrados cerrados anidados, cuyas diagonales se contraen en un punto, que definen un único punto en $\mathcal{Q}$, imagen de $f_h(t)$.

*Por tanto, se denomina **Curva de Hilbert** a $f_h(\mathcal{I})$.*

Teorema 7.2.1. *La aplicación $f_h : \mathcal{I} \longmapsto \mathcal{Q}$ está bien definida, es continua, sobreyectiva y no es diferenciable en ningún punto.*

Hilbert, en su definición, demostró que el proceso puede repetirse tantas veces como se quiera, siguiendo las siguientes dos premisas, hasta el punto de que el área de cada subcuadrado tiende a cero y entonces la curva ocupa toda la superficie rellenando todo el espacio.

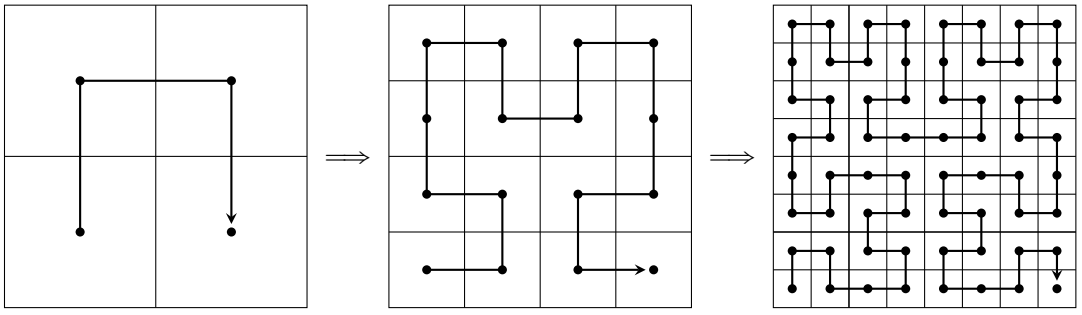
- Subintervalos adyacentes están en biyección con subcuadrados adyacentes.
- Un cuadrado que corresponde a un intervalo, al dividir este en subintervalos, estos se corresponden con subcuadrados contenidos en el cuadrado inicial, es decir, se preserva la relación de inclusión.

7.3. Definición y codificación personal

Hilbert, en su trabajo, describió la construcción de la curva con notación de números complejos para poder representar las transformaciones y formalizar las rotaciones de los subcuadrados para que sigan cumpliendo la propiedad de adyacencia entre particiones de cuadrados y intervalos consecutivos.

En este estudio, se ha hecho de forma distinta. Como no se quiere demostrar la formalidad de la función, sino implementarla algorítmicamente, se ha encontrado una recurrencia de patrones en la generación de la geometría. En este trabajo, se codifica la curva a partir de condiciones; es decir, según cómo esté orientada la curva en un cuadrado con cuatro centros, este se divide en un orden u otro de 4 subcuadrados con 4 centros cada uno.
Se ve mejor gráficamente:

Si se ven las primeras 4 iteraciones del algoritmo se obtienen las siguientes distribuciones:



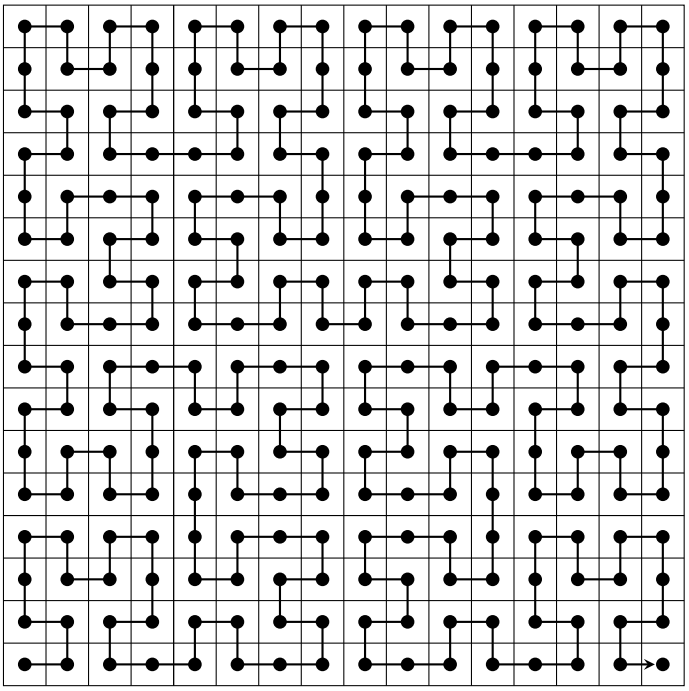


Figura 7.3: Primera, segunda, tercera y cuarta iteración de la curva de Hilbert. (Rodrigo Arbós. 2025)

Se puede identificar que todo subcuadrado dividido en cuatro centros solo tiene cuatro posibles ordenaciones posibles.
Y cada una de estas, en la siguiente iteración, se divide en cuatro ordenaciones nuevas que son combinaciones de las únicas cuatro posibles.

Gráficamente se pueden representar estas transformaciones de la siguiente forma:

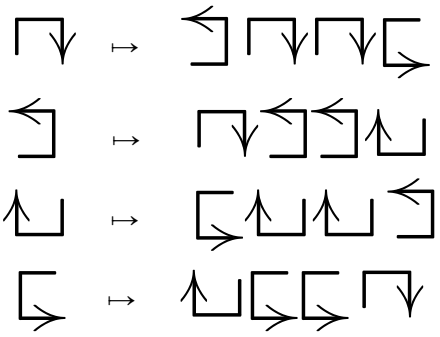


Figura 7.4: Transformación de la curva dentro del cuadrado según su orientación. (Rodrigo Arbós. 2026)

Cada una de las cuatro combinaciones resultantes está formada por los cuatro centros

nuevos que genera cada centro de la anterior combinación. Estos grupos de cuatro centros se ordenan siguiendo el orden de la anterior combinación; es decir, el orden en que se forman los cuatro subcuadrados nuevos va regido por el orden que tenía la curva en los cuatro centros en la anterior iteración.

Esto, para entender cómo se ha implementado computacionalmente, se puede representar de la siguiente manera:
Dado un cuadrado de cuatro centros a, b, c, d que se va a dividir en dieciséis subcuadrados, estos se pueden indexar de la siguiente forma:

b	c
a	d

$\Longrightarrow$

b_b	b_c	c_b	c_c
b_a	b_d	c_a	c_d
a_b	a_c	d_b	d_c
a_a	a_d	d_a	d_d

Figura 7.5: División y identificación de los subcuadrados para una nueva iteración. (Rodrigo Arbós. 2026)

Y es aquí, donde dependiendo de la condición dada por el orden que sigue la curva en los cuatro centros iniciales, se forma una combinación u otra en los dieciseis nuevos centros:

- $a\ b\ c\ d \longmapsto a_a\ a_d\ a_c\ a_b\ b_a\ b_b\ b_c\ b_d\ c_a\ c_b\ c_c\ c_d\ d_c\ d_b\ d_a\ d_d$
- $a\ d\ c\ b \longmapsto a_a\ a_b\ a_c\ a_d\ d_a\ d_d\ d_c\ d_b\ c_a\ c_d\ c_c\ c_b\ b_c\ b_d\ b_a\ b_b$
- $c\ d\ a\ b \longmapsto c_c\ c_b\ c_a\ c_d\ d_c\ d_d\ d_a\ d_b\ a_c\ a_d\ a_a\ a_b\ b_a\ b_d\ b_c\ b_b$
- $c\ b\ a\ d \longmapsto c_c\ c_d\ c_a\ c_b\ b_c\ b_b\ b_a\ b_d\ a_c\ a_b\ a_a\ a_d\ d_a\ d_b\ d_c\ d_d$

7.4. Código para la implementación

Una vez se entiende cómo funciona el algoritmo que dibuja la curva de Hilbert, tan solo hace falta plasmar dichas condiciones en el código. En este trabajo se ha usado Python y se puede encontrar el programa en la sección A.1.2 del apéndice. Allí se puede ver codificada la función que genera una lista de puntos ordenados que son las coordenadas por las que pasa la curva de Hilbert sobre un rectángulo de cualquier tamaño.

Para ejecutarla, se tiene que definir la anchura y la altura del rectángulo de la superficie a estudiar, así como indicar también el número de iteraciones del algoritmo de la curva de Hilbert que se desea dibujar en ella.

En los siguientes ejemplos se muestra cómo evaluar la función y el resultado de cada ejecución, la lista de centros ordenados por los que pasa la curva: 7.3.

```
# Parametros
x_max, y_max = 1,1
iteraciones = 1
# Generar centros
(centros,A_x,A_y) = centros_hilbert(x_max,y_max,iteraciones)
# Comprobar
print(centros)
```

Figura 7.6: Centros del rectángulo (1,1) con iteración 1

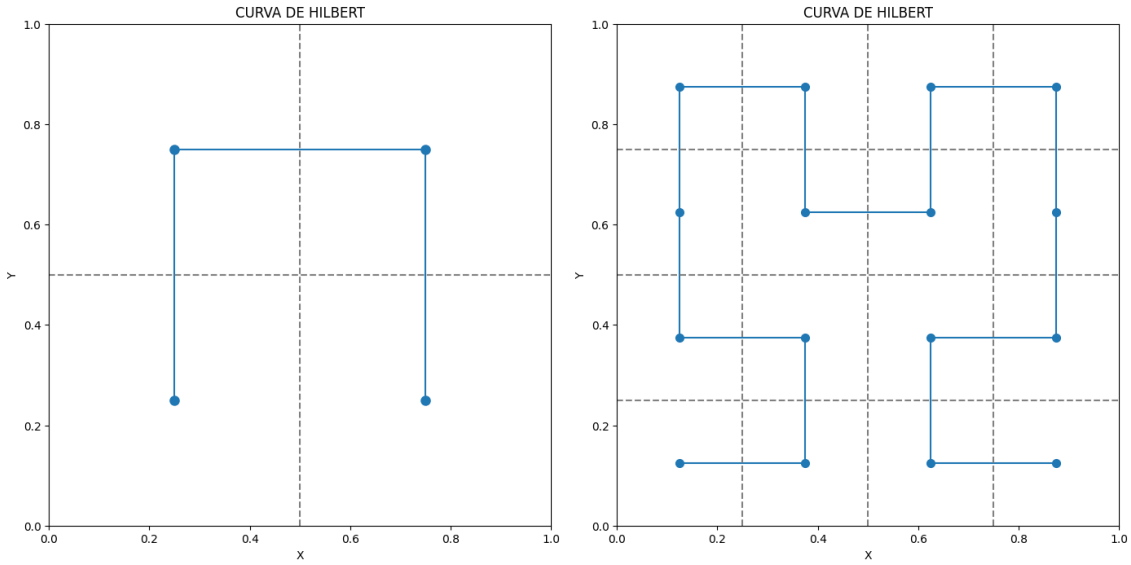
```
[(0.25, 0.25), (0.25, 0.75), (0.75, 0.75), (0.75, 0.25)]
```

```
# Parametros
x_max, y_max = 1,1
iteraciones = 2
# Generar centros
(centros,A_x,A_y) = centros_hilbert(x_max,y_max,iteraciones)
# Comprobar
print(centros)
```

Figura 7.7: Centros del rectángulo (1,1) con iteración 2

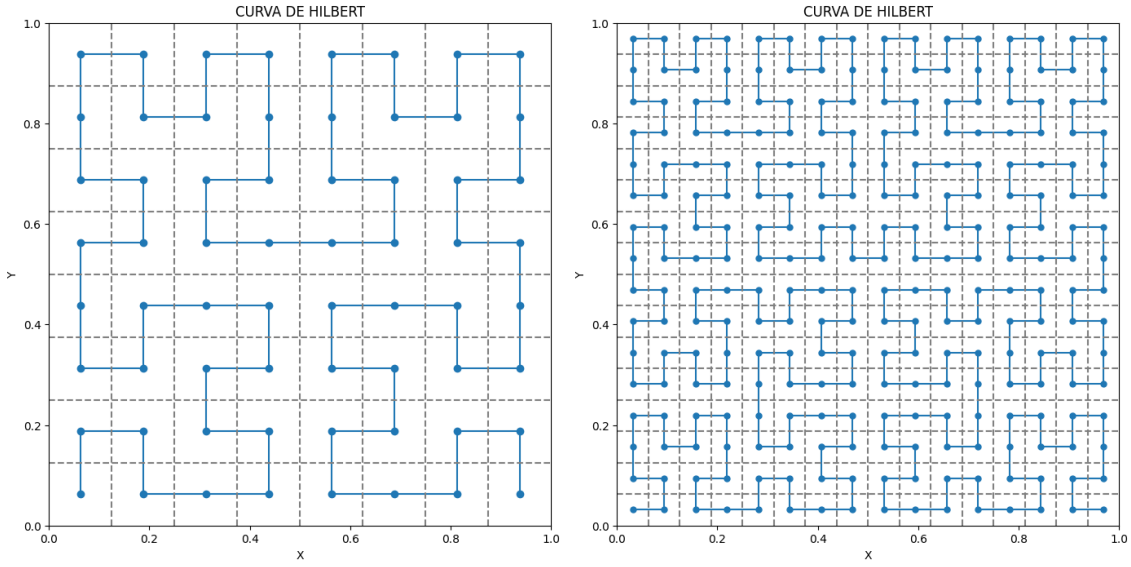
```
[(0.125, 0.125), (0.375, 0.125), (0.375, 0.375), (0.125, 0.375),
↪ (0.125, 0.625), (0.125, 0.875), (0.375, 0.875), (0.375, 0.625),
↪ (0.625, 0.625), (0.625, 0.875), (0.875, 0.875), (0.875, 0.625),
↪ (0.875, 0.375), (0.625, 0.375), (0.625, 0.125), (0.875, 0.125)]
```

Si con esta lista de puntos se ejecuta la función de la sección A.1.3, se pueden ver exactamente las representaciones que ya se han descrito anteriormente.



(a) Iteración 1 (b) Iteración 2

Figura 7.8: Representación de los ejemplos 7.6, 7.7 (Rodrigo Arbós. 2025)



(a) Iteración 3 (b) Iteración 4

Figura 7.9: Representación con computación de las últimas dos figuras de 7.3 (Rodrigo Arbós. 2025)

Ahora bien, como se ha explicado, esta función no solo trabaja sobre una superficie cuadrada perfecta. También permiten obtener la distribución de los puntos de la curva que rellena el espacio en rectángulos cuyos lados pueden ser diferentes entre sí.

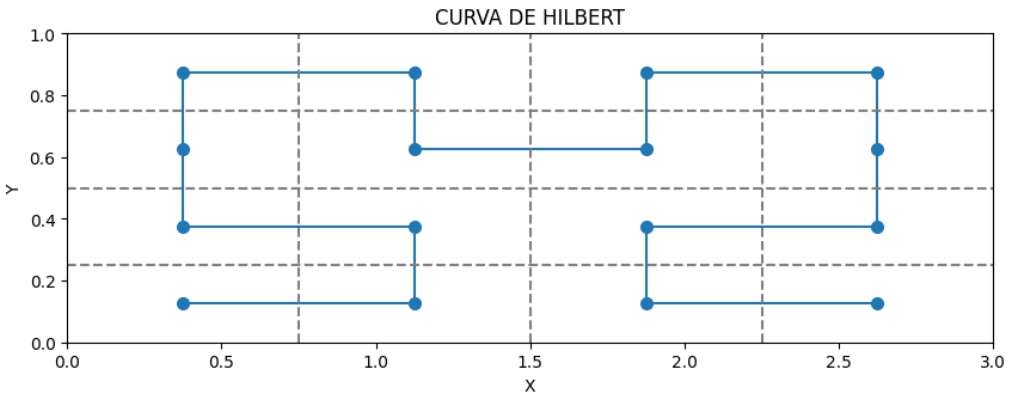


Figura 7.10: Curva de Hilbert sobre rectángulo (3,1) y 2 iteraciones. (Rodrigo Arbós. 2025)

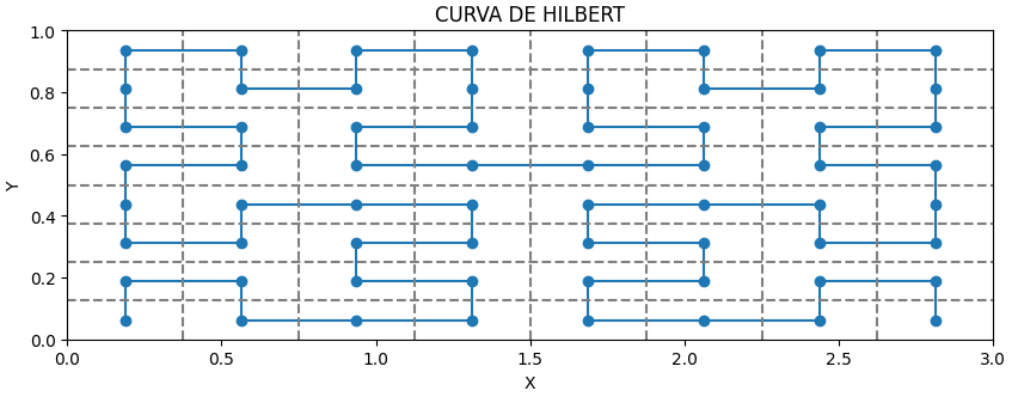


Figura 7.11: Curva de Hilbert sobre rectángulo (3,1) y 3 iteraciones. (Rodrigo Arbós. 2025)

En la siguiente figura se observa como se distribuye la curva de Hilbert con iteración 4, es decir, análoga a la última figura de la definición 7.3 o la representación (b) de 7.9, pero esta vez, representada sobre una superficie que mide de lado horizontal el triple que de lado vertical.

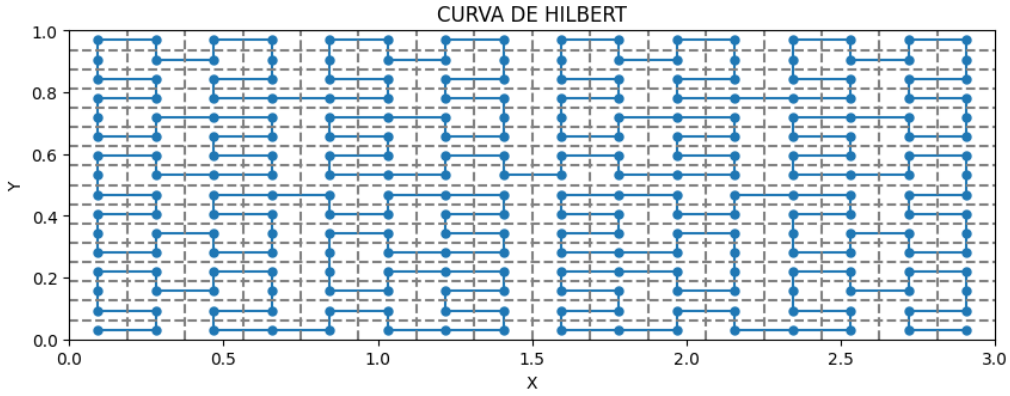


Figura 7.12: Curva de Hilbert sobre rectángulo (3,1) y 4 iteraciones. (Rodrigo Arbós. 2025)

En este ejemplo, el orden de los puntos se distribuye de la misma manera, pero el tamaño de las divisiones en cada subcuadrado es diferente, ya que siempre se mantiene la misma proporción que la superficie original.

Como se ha explicado y definido, la curva de Hilbert es una curva que rellena el espacio. Hasta ahora se han visto ejemplos y representaciones hasta la iteración 4, lo que proporciona una visión clara de cómo se genera la curva a partir de las iteraciones de la distribución inicial de los puntos.

Si se lleva el proceso computacional dos niveles más allá, se alcanza la iteración 6, la cual genera una lista de 4^6 puntos, es decir, 4096 nodos ordenados según el algoritmo descrito. Esta iteración refleja de manera mucho más precisa la idea de una curva que rellena el espacio.

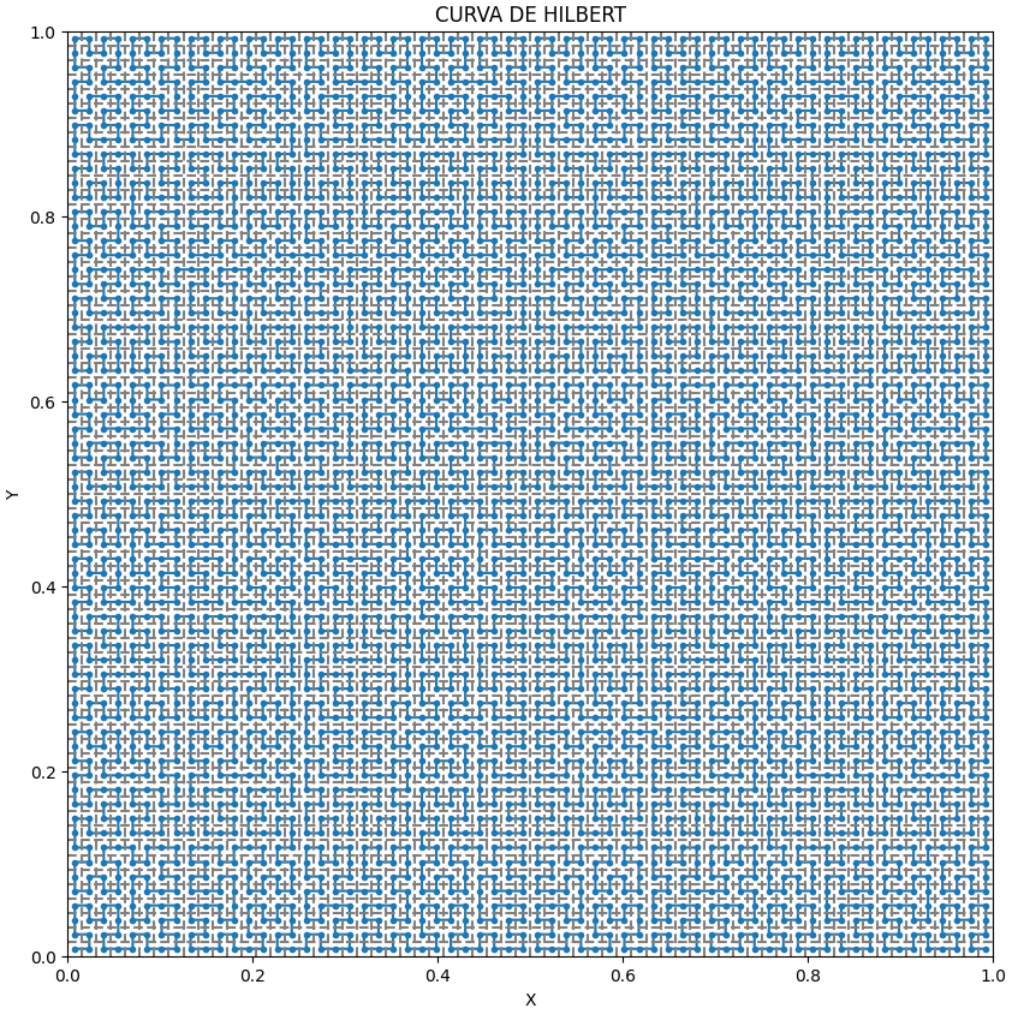


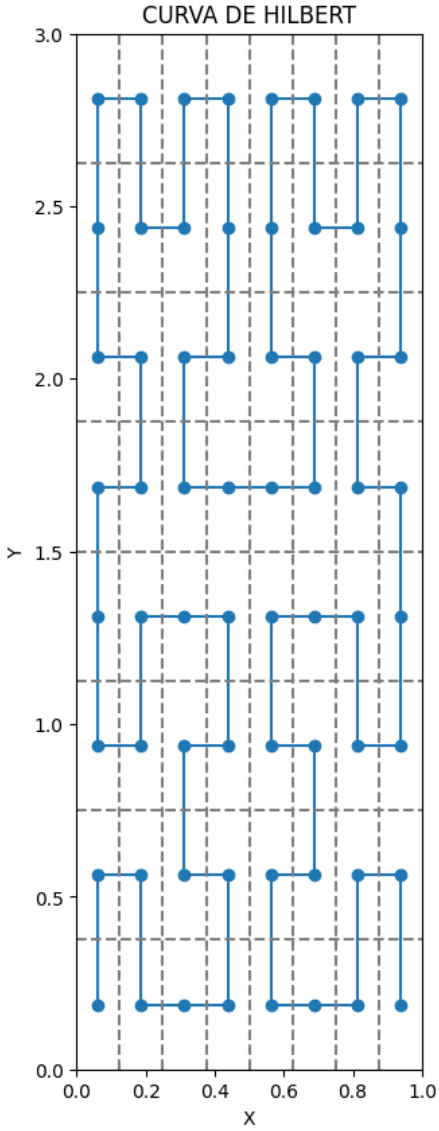
Figura 7.13: Curva de Hilbert sobre rectángulo (1,1) y 6 iteraciones. (Rodrigo Arbós. 2025)

En este ejemplo, se observa que la geometría sigue siempre el mismo algoritmo para transformar una iteración en la siguiente. Pero esta vez, la representación final, tiene un mayor numero de nodos calculados. Para la iteración 6, ya se aprecia una superficie en la que la curva ocupa prácticamente todo el espacio, dejando muy pocas zonas libres.

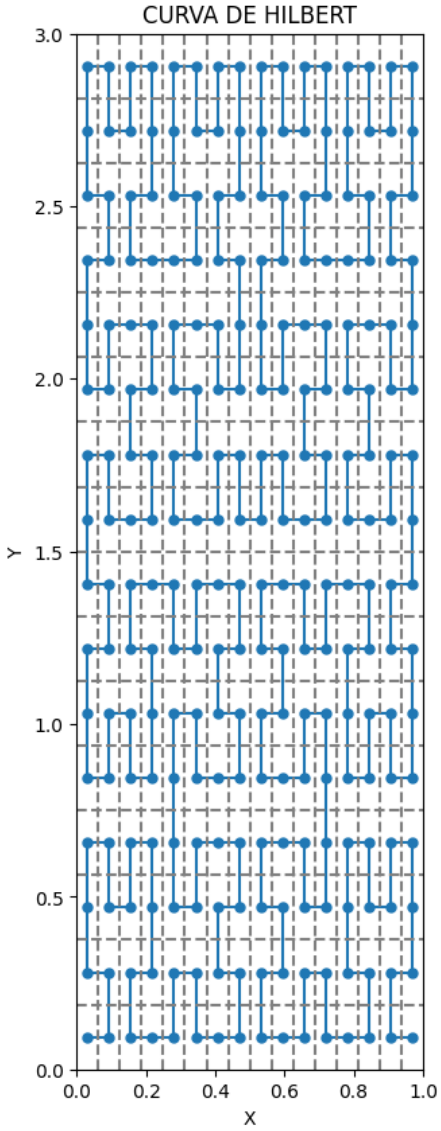
Finalmente, se pueden visualizar nuevas representaciones de la curva de Hilbert, pero esta vez sobre rectángulos más altos que anchos.

No se deben confundir con los ejemplos que se observan en las figuras 7.11 o 7.12 giradas 90°, es decir, representadas en vertical.

En las siguientes ejecuciones, los puntos se ordenan de la misma forma y siguen el mismo recorrido que en las anteriores figuras. Pero para este caso, se disponen sobre rectángulos que mantienen la proporción del original, es decir, con una altura triple que la anchura.



(a) Iteración 3



(b) Iteración 4

Figura 7.14: Curva de Hilbert sobre rectángulo (1,3). (Rodrigo Arbós. 2025)

7.5. Conclusión

Hoy en día, la curva de Hilbert se ha vuelto muy relevante no solo en matemáticas puras, sino también en áreas aplicadas como compresión de imágenes, estructuras de datos espaciales y computación gráfica.
Ahora el objetivo del TFM es también darle importancia y utilidad como estructura de transferencia térmica.

En el apéndice final, sección A.4, se ha ampliado el código para que no solo genere la curva representativa, sino que también se visualice la geometría de la tubería por la cual circula el fluido dibujando así lo que serían las paredes del canal.

8. Transferencia térmica sobre la curva de Hilbert

El objetivo de este capítulo es definir la forma en que el fluido pierde o gana calor por la tubería, es decir, formalizar el intercambio de energía que lleva al equilibrio térmico dentro de la curva de Hilbert.

Se parte de una lista de puntos que indican el recorrido de la tubería por la que circula el fluido.

Definición 15. *La tubería que participa en el proceso de transferencia térmica, es decir, el recorrido del fluido, se puede definir como una discretización de puntos*

$$p_0, p_1, \cdots, p_N$$

Estos, en el caso de la curva de Hilbert, son los centros de cada rectángulo ordenados según el recorrido que sigue la estructura. Cada p_i es el elemento i-ésimo de la lista que se obtiene al aplicar la función A.1.2 y que se puede ver, por ejemplo, en la sección 7.6.

En este estudio, siguiendo la teoría previa explicada, se puede afirmar que se tiene convección forzada. Este movimiento del fluido por la tubería, que se ha definido como *transporte de masa*, permite formalizar el paso de temperatura de un punto al siguiente del recorrido.

En conclusión, siguiendo la notación de tubería como una sucesión de puntos $p_0, p_1, \cdots, p_N$, se puede definir lo siguiente:

Definición 16. *Se denota* ***la temperatura del fluido*** *que circula por una tubería* ***en un punto*** p_i ***a tiempo*** $j \in \mathbb{N}$ *de la siguiente forma:*

$$T_i^j$$

Donde $j \in \mathbb{N}$ representa el índice de la variable temporal discretizada.

El tiempo, pero, no se implementa como tal, sino que esta discretización será la forma en la que se indicará cuántos bucles computacionales del código se han completado hasta llegar al equilibrio.

Siguiendo con la notación, se formaliza el transporte de masa, es decir, el proceso de cómo la temperatura del fluido avanza por la tubería $p_0, p_1, \cdots, p_N$ como:

$$T_{i+1}^{j+1} = T_i^j \tag{8.1}$$

Aunque a esta recurrencia faltaría añadirle el intercambio energético por conducción, convección y radiación, que son los intercambios de energía que permitirán el equilibrio térmico sobre todo punto p_i de la tubería.

8.1. Transferencia térmica en el punto p_i

En esta sección, se formaliza la transferencia térmica del fluido mientras este avanza por la tubería. Este intercambio de energía dependerá del orden que sigan los puntos p_i respecto a la posición del plano, ya que dependiendo del lugar en el que esté, habrá unas u otras interacciones entre los otros centros.

En un punto p_i de la tubería, se pueden definir los siguientes elementos:

- p_i : Nodo i de la tubería en el que se encuentra el fluido y que interviene en la transferencia térmica.
- T_i^j : Temperatura del fluido en el nodo i en la recurrencia j del cálculo computacional.
- T_{int} : Temperatura con la que entra el fluido a la tubería; esta es constante durante todo el proceso.
  Se puede afirmar entonces que $T_0^n = T_{int} \quad \forall n \in \mathbb{N}$.

En cuanto a la temperatura, se sabe que avanza por el canal por el transporte de masa, pero antes de pasar al siguiente punto p_{i+1} habrá que aplicarle la conducción axial de los nodos adyacentes pero no consecutivos en el recorrido, la pérdida o ganancia de calor por radiación con la temperatura exterior y los procesos de convección.

- T_{ext} : Temperatura del exterior constante que participa en el proceso de radiación.

Esta recurrencia de temperatura define el proceso en que los nodos llegan al equilibrio térmico, es decir, al régimen estacionario.

Si se estudia bien cada nodo de la tubería, este no solo va a representar un punto del plano, sino que realmente indicará un volumen que representa el espacio que ocupa la tubería. Para facilitar los cálculos, se va a representar la tubería de forma rectangular, es decir, se simulará un canal con forma de prisma rectangular.
Siguiendo con estos conceptos, sobre cada uno de los nodos habrá unas constantes que participarán en el proceso:

- Δx : Longitud hasta el siguiente nodo.

- A_f : Sección del fluido que circula por la tubería.
- A_{lat} : Área de la superficie de la cara lateral de la tubería.
- A_{sup} : Área de la superficie de la cara superior o inferior de la tubería.

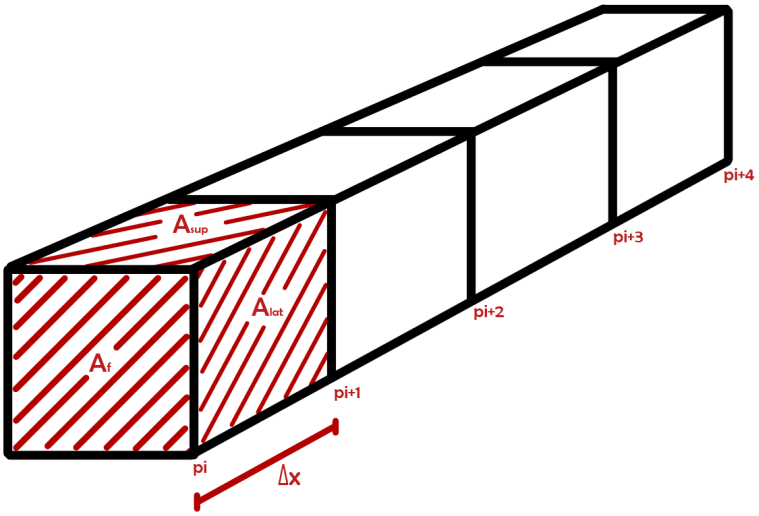


Figura 8.1: Variables que determinan el canal del fluido. (Rodrigo Arbós. 2026)

Se puede definir entonces la masa del fluido que pasa por un nodo como

$$m = \rho \; A_f \; \Delta x \tag{8.2}$$

- ρ : Densidad del fluido.

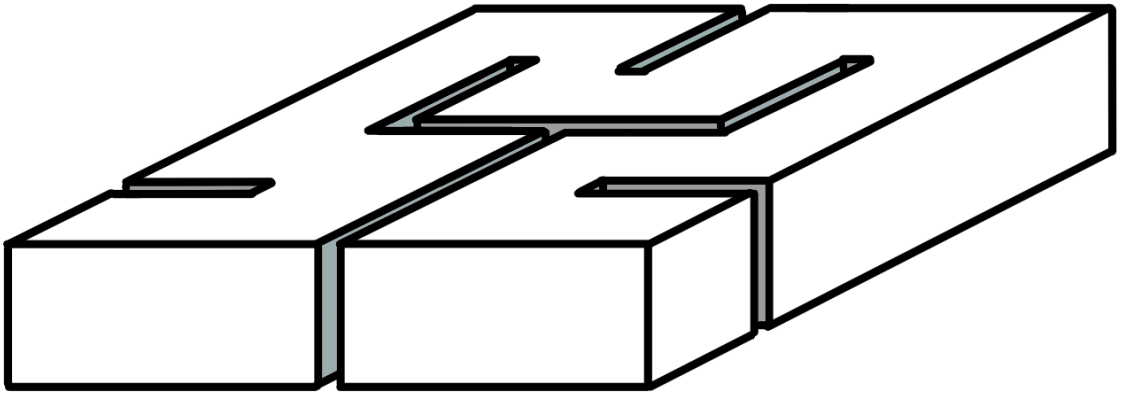


Figura 8.2: Tubería de Hilbert con estructura rectangular. (Rodrigo Arbós. 2026)

Se hace hincapié que para facilitar cálculos y comprobación de los resultados se considera la tubería rectangular en un espacio tridimensional que llena la superficie, pero rodeada por la parte superior e inferior del mismo material. Es decir, tendrá convección exterior y radiación dobladas, ya que estos intercambios estarán presentes por las caras superiores e inferiores del canal.

Se añade que se denomina ***vóxel*** a la caja que define un punto en la tubería. Es decir, un vóxel es aquel trozo de canal indicado por un nodo.

8.1.1. Balance de energía

Se ha visto que la transferencia térmica se puede expresar como:

$$Q = m\ c_p\ \Delta T$$

$$Q = Q_{conv.} + Q_{cond.} + Q_{rad.}$$

Si se igualan estas dos definiciones y se desarrollan los términos con las expresiones 6.4, 6.3, 6.5, se puede terminar de definir el transporte de masa 8.1 que se había iniciado y solo faltaba añadirle los términos de intercambio de energía que permiten llegar al equilibrio térmico.

$$\begin{aligned} m\ c_p\ \frac{dT_i}{dt} = & \underbrace{h_{int}\ A_f\ (T_{i-1}^n - T_i^n)}_{\text{convección interna}} + \underbrace{2\ h_{ext}\ A_{sup}\ (T_{ext} - T_i^n)}_{\text{convección externa}} \\ & + \underbrace{\sum_{j \in \mathcal{N}(i)} U_{lat}\ A_{lat}\ (T_j^n - T_i^n)}_{\text{conducción}} - \underbrace{2\ \epsilon\ \sigma\ A_{sup}\ (T_i^4 - T_{ext}^4)}_{\text{radiación}} \end{aligned} \tag{8.3}$$

Seguidamente, se aclara, a partir de la teoría de la introducción, cada término de la ecuación.

8.1.2. Convección interna

Término que indica el intercambio de temperatura entre puntos consecutivos de la tubería. Este se calcula siempre comparando la temperatura del nodo actual con la del anterior.

$$h_{int}\ A_f\ (T_{i-1}^n - T_i^n)$$

- Si el fluido viene más caliente $\longrightarrow T_i$ sube.
- Si el fluido viene más frío $\longrightarrow T_i$ baja.

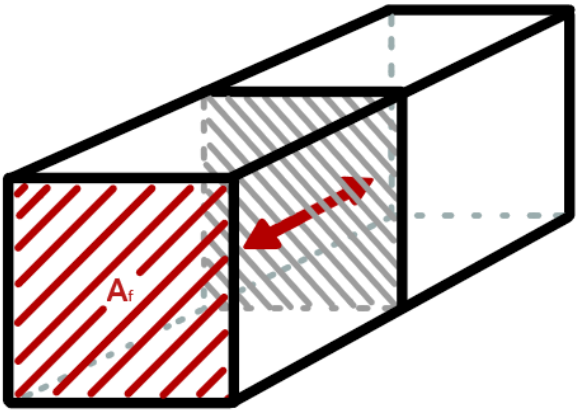


Figura 8.3: Esquema de la convección interna en la simulación. (Rodrigo Arbós. 2026)

8.1.3. Convección externa

Término que indica el intercambio de temperatura entre el fluido y el cuerpo que envuelve la tubería. Como se considera que la tubería rellena todo el espacio y está rodeada del mismo material, solo habrá convección externa por la capa superior e inferior. Además, como las dos capas tienen la misma área se pueden agrupar los términos obteniendo el término de la ecuación.

$$2\ h_{ext}\ A_{sup}\ (T_{ext} - T_i^n)$$

- Si el fluido está más caliente que el exterior $\longrightarrow$ T_i sube y el punto pierde calor.
- Si el fluido está más frío que el exterior $\longrightarrow$ T_i baja y el punto gana calor.

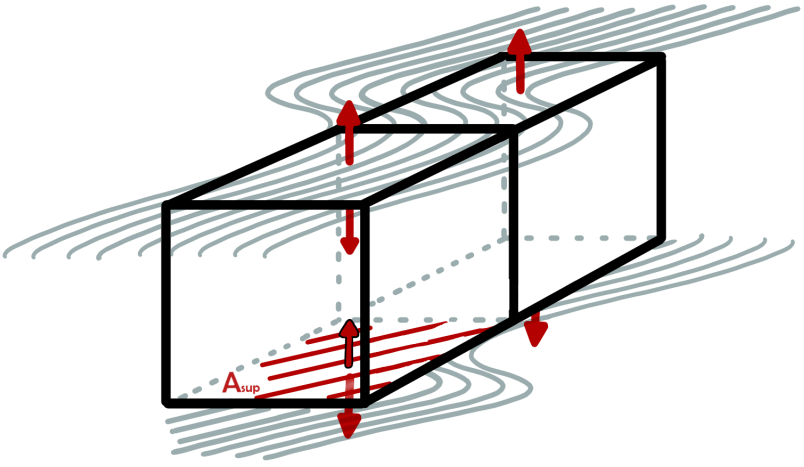


Figura 8.4: Esquema de la convección externa en la simulación. (Rodrigo Arbós. 2026)

8.1.4. Conducción

Término que indica el intercambio de temperatura entre fluidos adyacentes.

$$U_{lat}\ A_{lat}\ (T_j^n - T_i^n)$$

Representa que si el nodo j colindante con el nodo i pero no consecutivo sobre el recorrido tiene más temperatura ($T_j > T_i$), entonces el nodo i gana calor del nodo j. Si no, por lo contrario se tiene ($T_j < T_i$), se transmite energía del nodo i al nodo j y por tanto el nodo i pierde calor.

- U_{lat} : Coeficiente global de transferencia de calor entre tuberías colindantes.
  Como se ha visto en la teoría previa, la ecuación 6.3 representa la constante como k; así pues, se hace lo mismo para no perder la notación. k será la **constante de conducción**.
- A_{lat} : Área de la parte de la tubería que interacciona con otra tubería y produce esta conducción.

$$k\ A_{lat}\ (T_j^n - T_i^n)$$

Pero este intercambio para un nodo i se tiene que calcular para todos sus nodos adyacentes no consecutivos; entonces, se aplica:

$$\sum_{j \in \mathcal{N}(i)} k \cdot A_{lat} \cdot (T_j^n - T_i^n)$$

Donde $\mathcal{N}(i)$ es el conjunto de nodos adyacentes al punto p_i pero no consecutivos del recorrido del fluido.

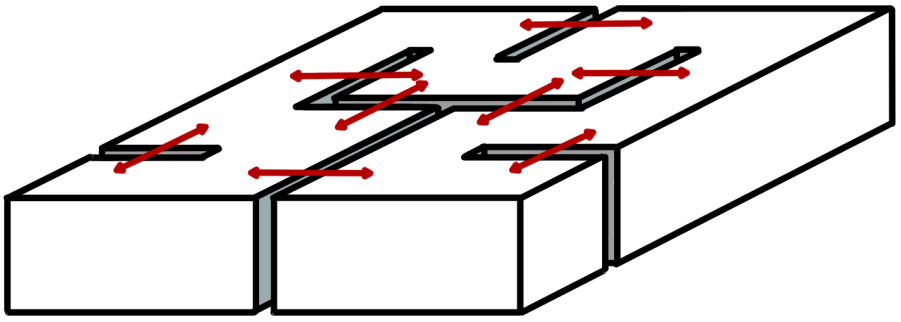


Figura 8.5: Esquema de la conducción en la simulación. (Rodrigo Arbós. 2026)

8.1.5. Radiación

Término que indica el intercambio de temperatura de la tubería hacia el exterior, el cual tiene una temperatura constante.

$$\epsilon \ \sigma \ A_{sup} \ ({T_i}^4 - {T_{ext}}^4)$$

- Si $T_i > T_{ext} \longrightarrow$ entonces el fluido pierde calor en el nodo i.
- Si $T_i < T_{ext} \longrightarrow$ entonces el fluido gana calor en el nodo i.

Pero al igual que se ha hecho en el anterior apartado, la ecuación 6.5 tiene una única constante q; pues se agrupan las constantes ϵ y σ en un único coeficiente q, el **coeficiente de radiación**, y así se obtiene el término análogo a la teoría previa.

$$q \ A_{sup} \ ({T_i}^4 - {T_{ext}}^4)$$

Ahora bien, como se ha explicado en la sección 8.1.3, el canal estará envuelto del mismo material por la cara superior e inferior, es decir, envuelto por una temperatura exterior constante. Así pues, de la misma manera que en dicha sección, se dobla el término para representar el intercambio de temperatura por las dos caras.

$$2 \ q \ A_{sup} \ ({T_i}^4 - {T_{ext}}^4)$$

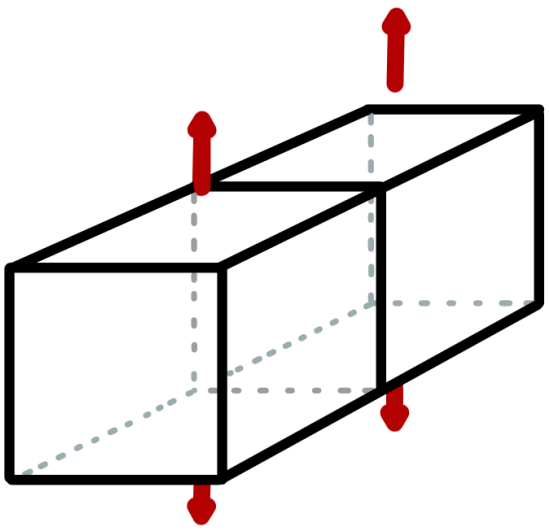


Figura 8.6: Esquema de la radiación en la simulación. (Rodrigo Arbós. 2026)

8.2. Recurrencia de la transferencia térmica

Despues de aprofundizar y detallar cada término que participa en el proceso de transferencia térmica se puede seguir desarrolando la ecuación 8.3 y llegar a la recurrencia que se busca.

1. Se aproxima $\frac{dT_i}{dt}$ por diferencia finita progresiva obteniendo

$$\frac{T_i^{n+1} - T_i^n}{\Delta t}$$

2. Se aisla de la ecuación el término T_i^{n+1}.

$$T_i^{n+1} = T_i^n + \frac{\Delta t}{m\ c_p}[h_{int}\ A_f\ (T_{i-1}^n - T_i^n)\ + \\ 2\ h_{ext}\ A_{sup}\ (T_{ext} - T_i^n) + \sum_{j \in \mathcal{N}(i)} k\ A_{lat}\ (T_j^n - T_i^n) - 2\ q\ A_{sup}\ (T_i^4 - T_{ext}^4)]$$

Pero como se ha explicado, no se va a discretizar el tiempo, sino que el índice de la variable temporal indicará el número de iteraciones completadas del cálculo computacional. Una característica de este término es que no va a tener límite superior, sino que se detendrá al llegar al equilibrio térmico.

Definición 17. *La **recurrencia de la temperatura del punto de la tubería** p_i viene dada por la iteración:*

$$T_i^{n+1} = T_i^n + \frac{\Delta t}{m\ c_p}[h_{int}\ A_f\ (T_{i-1}^n - T_i^n)\ + \\ 2\ h_{ext}\ A_{sup}\ (T_{ext} - T_i^n) + \sum_{j \in \mathcal{N}(i)} k\ A_{lat}\ (T_j^n - T_i^n) - 2\ q\ A_{sup}\ (T_i^4 - T_{ext}^4)] \tag{8.4}$$

Donde:

- T_i^n : Temperatura del nodo i en la iteración n del cálculo computacional.
- m : Masa del fluido.
- c_p : Calor específico del fluido.
- Δt : Intervalo de tiempo entre dos instantes consecutivos de la recurrencia.
- $\mathcal{N}(i)$: Conjunto de nodos colindantes con el nodo i pero no consecutivos.

$$\mathcal{N}(i) := \{j \neq i \ \text{ tal que } \ |p_i - p_j| = \Delta x \ \text{ y } \ j \notin \{i-1, i+1\}\}$$

- A_{lat} : Área lateral de la tubería.
- A_{sup} : Área superior de la tubería.
- h_{int} : Coeficiente de transferencia térmica del proceso de convección interna.
- h_{ext} : Coeficiente de transferencia térmica del proceso de convección externa.
- k : Coeficiente de transferencia térmica del proceso de conducción.
- q : Coeficiente de transferencia térmica del proceso de radiación.

8.2.1. Determinación de los parámetros del modelo

En el algoritmo hay múltiples constantes que, dependiendo de la situación, pueden tomar unos valores u otros.

En cuanto a las constantes térmicas de los mecanismos de transferencia de calor, se ha recurrido a un libro que agrupa diferentes valores para identificar qué término requiere cada situación, (Incropera, DeWitt, Bergman, y Lavine, 2007).

Radiación Para la constante de su término q, al tratarse de un producto de dos valores y uno estar definido por teoría, la constante de Stefan-Boltzmann que tiene valor $\sigma = 5,670 \cdot 10^{-8}\ \frac{W}{m^2K^4}$. Falta definir el parámetro ϵ, que es un valor entre 0 y 1 que determina cómo se comporta el material para la radiación.
En este trabajo se usará $\epsilon = 0{,}85$, reflejando que es un fluido con buena radiación, pero que no se comporta como un cuerpo negro.

$$\boxed{q = \epsilon\ \sigma = 0{,}85 \cdot 5{,}670 \cdot 10^{-8} = 4{,}819 \cdot 10^{-8} \frac{W}{m^2K^4}}$$

Conducción Siguiendo con la ayuda del libro mencionado, viendo ejemplos donde se calcula la energía transferida entre tuberías, si se considera la tubería como un panel radiante de cobre, tendría una constante $k = 400\frac{W}{mK}$, pero eso sucede si las tuberías están directamente en contacto sin ningún material por medio. Como en realidad nunca estarán los canales en contacto directo, se tiene que reducir este valor.

$$\boxed{k = 200\frac{W}{mK}}$$

Convección Como se ha explicado, hay que diferenciar entre dos procesos y, para cada uno, adaptar una constante diferente. Para ello se aplica la siguiente tabla del libro:

Tabla 8.1: Coeficientes de convección en transferencia térmica.

Proceso	h **(W/m^2·K)**
Convección libre	
Gases	2–25
Líquidos	50–1000
Convección forzada	
Gases	25–250
Líquidos	100–20000

Para el proceso interno, como se trata de una convección forzada al mover el fluido por el canal y es un intercambio entre el mismo líquido, se asigna un valor promedio, pero no muy alto, ya que el material tampoco viaja tan rápido como para tener un coeficiente elevado.

$$\boxed{h_{int} = 1000 \frac{W}{m^2 K}}$$

En cuanto a la convección externa, como es un proceso libre, es decir, no forzado, al ser un intercambio del líquido con el aire exterior, se asigna un valor entre 2 y 15.

$$\boxed{h_{ext} = 11 \frac{W}{m^2 K}}$$

El calor específico depende del fluido que circule por la tubería y, para cada material, al igual que la constante de Stefan-Boltzmann, ya está calculado y es invariable.
Por ejemplo, el agua tiene un calor específico de $c_p = 4180 \frac{J}{kgK}$.
Sin embargo, para determinar $\frac{\Delta t}{m}$ no es tan sencillo, ya que, si se quiere ser riguroso en las comparaciones, se tendrá que adaptar en cada caso este cuociente para poder realizar un buen análisis de los resultados.

El caudal en las tuberías de un circuito de refrigeración suele estar entre $0,5$ y $2,0$ L/min. En este estudio se definirá un caudal de $1,5$ L/min.
Físicamente, esto significa que

$$1,5 \ \frac{L}{min} \cdot \frac{0,001 \ m^3}{1 \ L} \cdot \frac{1 \ min}{60 \ s} \approx 2,5 \cdot 10^{-5} \ m^3/s = \dot{V}$$

Por tanto, el **caudal volumétrico $\dot{\mathbf{V}}$** que se quiere que sea constante para comparar rigurosamente las simulaciones será de $2,5 \cdot 10^{-5} \ m^3/s$ en toda simulación del estudio.

La tubería simulada, como ya se ha dicho, será rectangular, es decir, la sección tendrá forma de rectangulo con altura constante, pero el ancho y el largo dependerán de la discretización de la superficie según la curva que se dibuje.
Como esta depende de la iteración de Hilbert, situa los valores en unas medidas u otras dependiendo del caso.

Para asegurar esta constante se trabajará con caudal másico y así evitar los m^3 que rigen tener una distancia constante.

El **caudal másico $\dot{\mathbf{m}}$** representa la masa que circula por la tubería por segundo, es decir, kg/s.

$$\dot{\mathbf{m}} = \rho\ \dot{\mathbf{V}}$$

Donde ρ es la densidad del fluido que circula y varia según el material que fluye.

Si, por ejemplo, se escoge agua, $\rho = 1000\ kg/m^3 = 1\ kg/L$ y, por tanto, se obtendría un caudal másico de:

$$\dot{\mathbf{m}} = \rho\ \dot{\mathbf{V}} = 2,5 \cdot 10^{-5}\ \frac{m^3}{s}\ 1000\ \frac{kg}{m^3} = 0,025\ kg/s$$

Como se quiere, pues, $\dot{\mathbf{m}}$ constante, equivale a tener $\dot{\mathbf{m}} = \frac{m}{\Delta t}$ constante también. Esta forma de expresar el caudal másico es como se calcula normalmente y es, literalmente, su definición: la masa que pasa por un nodo en un intervalo de tiempo.
Ya que la masa del fluido, m, viene definida, como se ha visto en 8.2, por $\rho\ A_f\ \Delta x$, esta, según la iteración del algoritmo al crear la curva, tendrá un valor u otro, ya que variarán tanto Δx como A_f. Es por eso que, para mantener $\dot{\mathbf{m}}$ constante y ser rigurosos en las comparaciones de los resultados, habrá que adaptar Δt según la masa que se utilice.

En conclusión, Δt no será un parámetro cualquiera, sino que vendrá dado por la siguiente ecuación:

$$\Delta t = \frac{m}{\dot{\mathbf{m}}}$$

Que se puede desarrollar y se obtiene:

$$\Delta t = \frac{m}{\dot{\mathbf{m}}} = \frac{\rho\ A_f\ \Delta x}{\rho\ \dot{\mathbf{V}}} = \frac{A_f\ \Delta x}{\dot{\mathbf{V}}}$$

Así, de esta manera, al adaptar Δt para cada caso, se tendrá un caudal másico $\dot{\mathbf{m}}$ constante para poder hacer rigurosamente las comparaciones de los resultados.

9. Codificación

Una vez que se tiene toda la base teórica, se tienen todas las herramientas para codificar, como se ha hecho anteriormente en la sección 7.4, el algoritmo que permitirá alcanzar el objetivo principal. Partiendo de la recurrencia desarrollada en 17 y del camino resultante de la sección ya mencionada, se puede implementar la función que itera sobre todos los puntos ordenados de la superficie y así resolver el problema principal.
Antes, se introducirán dos funciones, una para enlazar las coordenadas a un valor númerico para representar la temperatura y otra que permite ver como se distribuye la temperatura que circula por el canal por toda la superficie.

9.1. Dibujar temperatura

Una vez que se tiene la lista de centros, resultado de la sección 7.4, se ejecuta la función que se puede encontrar en A.1.4, que crear un diccionario de Python y asocia cada nodo a una temperatura.
Con ello, cada punto del recorrido no solo indica una posición, sino que tiene asociado un valor numérico que representa la temperatura del fluido en el vóxel correspondiente.

Una vez creada la variable diccionario que representa la tubería térmica, se pueden ejecutar las funciones que se encuentran en A.1.5 para representar cada situación del problema. Esta función utiliza la librería *matplotlib* para crear primero un rectángulo y representar la superficie del estudio. Segundo, sobre cada coordenada dada por el diccionario, se representa la temperatura coloreando la región del punto con un tono más cálido o más frío según su proximidad a la temperatura máxima o mínima.

El objetivo principal es representar una situación real de transferencia térmica de un panel radiante expuesto al aire. Se considerará pues que el fluido siempre entra a una temperatura máxima de 55 grados.

$$\boxed{T_0^n = 55^\circ \ \forall n}$$

y que la temperatura exterior es de 17 grados constantes.

$$\boxed{T_{ext} = 17^\circ}$$

Si se ejecutan las funciones que se acaban de describir con estos valores, se obtienen las siguientes figuras que representan la situación inicial del problema:

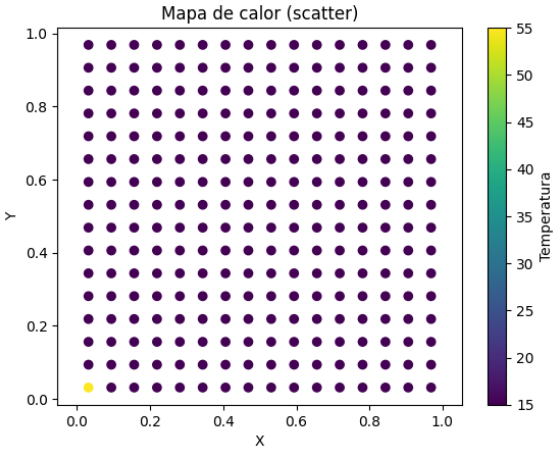
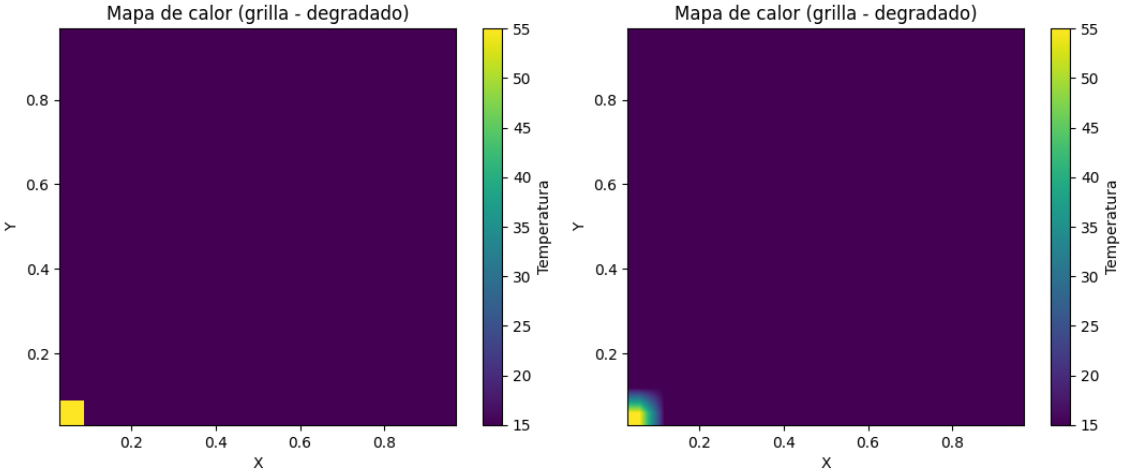


Figura 9.1: Situación inicial de la temperatura de los nodos. (Rodrigo Arbós. 2026)



(a) aspect = equal

(b) interpolation = bilinear

Figura 9.2: Situación inicial sobre toda la superficie. (Rodrigo Arbós. 2026)

9.2. Codificar los términos de la recurrencia

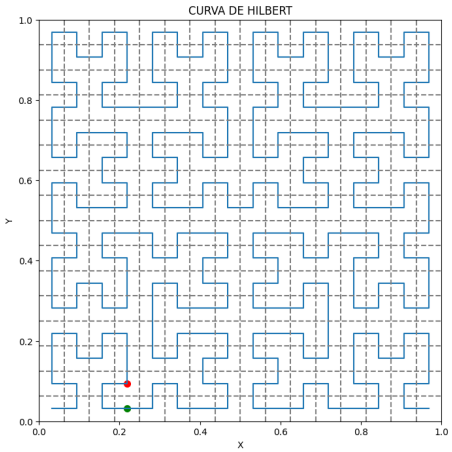
Para seguir codificando la simulación, se implementa cada término de la recurrencia por separado. Estos se pueden encontrar en la sección A.1.6, donde se ven las cuatro funciones que simbolizan los distintos mecanismos de transferencia térmica del algoritmo.

Remarcar que, para el cálculo del calor perdido por conducción, se necesita una lista de los nodos adyacentes, pero no consecutivos según el orden del canal.
Para ello, se crea la función A.1.7, que devuelve una lista de los vecinos no consecutivos para poder aplicar la función de conducción sobre cada uno de ellos.

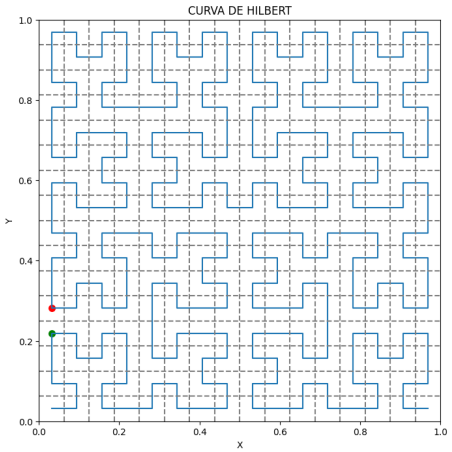
Se puede ver su funcionamiento en las siguientes imágenes, donde el punto verde re-

presenta el nodo actual y los puntos rojos representan los nodos que devuelve la función y sobre los que se aplicará el intercambio por conducción.

Se encuentran representadas las cuatro posibles situaciones.

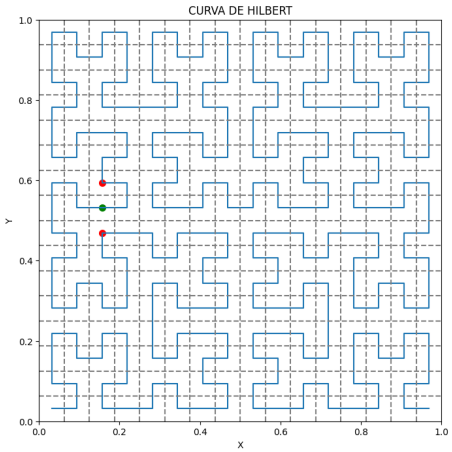


(a) Punto sobre lado horizontal

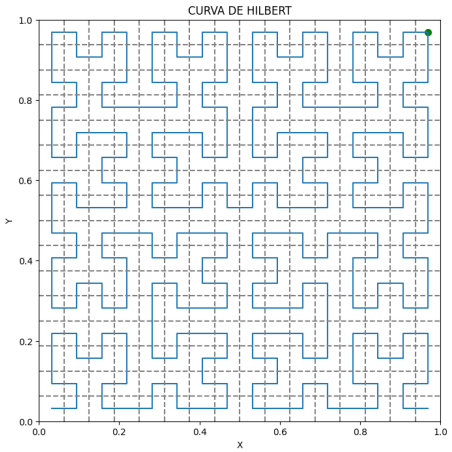


(b) Punto sobre lado vertical

Figura 9.3: Representación de vecinos. (Rodrigo Arbós. 2026)



(a) Punto con máximo de vecinos



(b) Punto sin vecinos

Figura 9.4: Representación de vecinos. (Rodrigo Arbós. 2026)

9.3. Codificación del algoritmo principal

Ahora que se tiene cada término codificado, solo hace falta generar una función con todas las constantes, inicializar las variables correspondientes dentro y que esta ejecute un bucle que no se detenga hasta llegar al equilibrio térmico. Esta condición se consigue

imponiendo que la diferencia de la temperatura de cada nodo con la temperatura en su bucle anterior sea más pequeña que una tolerancia. Esta variable se identificará como *eps* y se le asigna un valor bastante pequeño.

$$\boxed{eps = 1e - 3 = 1 \cdot 10^{-3}}$$

Este código se puede encontrar en la sección A.1.8 y, como resultado, devuelve el diccionario que representa la tubería con la temperatura en equilibrio térmico.

En el siguiente enlace, para comprender el transporte de masa de la simulación, se ha ejecutado la función con todos los coeficientes de transferencia a 0. De esta manera, se puede visualizar la simulación sin perdida de calor y ver como la tubería se va calentando siguiendo el recorrido de la curva de Hilbert.

https://youtu.be/3Bfj06ELW6U

Figura 9.5: Transporte de masa por Hilbert con iteración 4. (Rodrigo Arbós. 2026)

Finalmente se muestra la gráfica de la tubería resultante para este caso, donde se muestra la superficie completa con temperatura del líquido entrante por el vóxel inicial.

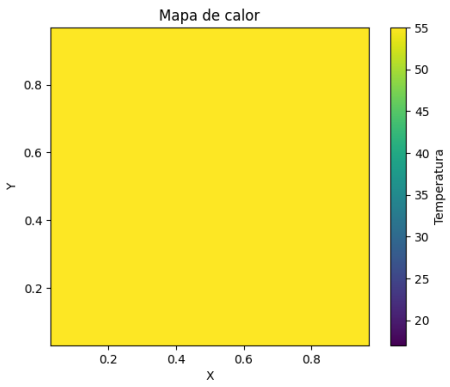


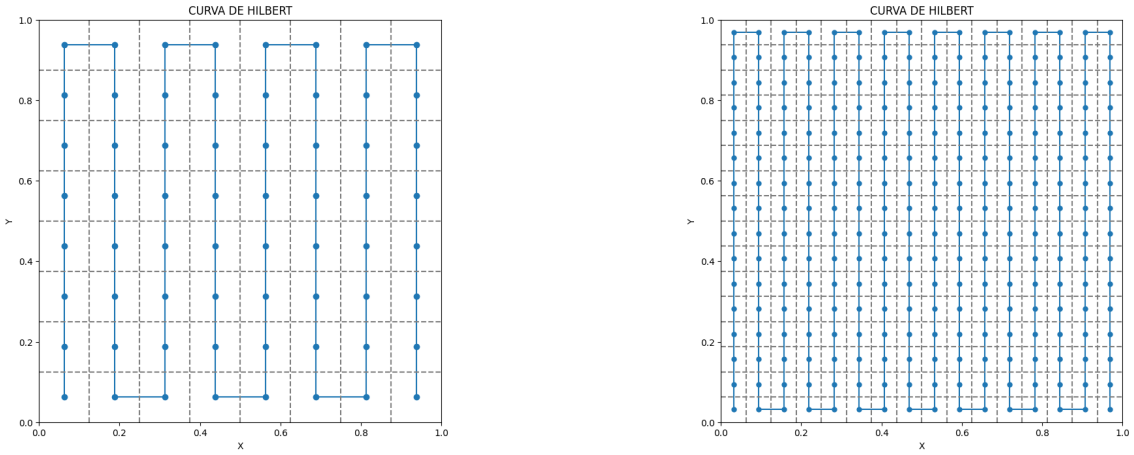
Figura 9.6: Resultado de la simulación sin perdida de calor. (Rodrigo Arbós. 2026)

9.4. Tubería de serpentín

Para finalizar esta sección, se describe brevemente el código que genera la geometria en serpentina y así poder realizar la comparación. Se ha implementado el algoritmo que genera el recorrido en la Sección A.2.

Esta función está diseñada de manera que, al ejecutarse con el mismo número de iteraciones que la curva de Hilbert, discretiza el rectángulo de forma equivalente y genera un recorrido serpenteante que comienza y finaliza en los mismos puntos que la curva fractal.

La función puede ejecutarse de forma análoga a los ejemplos de la Sección 7.4, y así visualizar el comportamiento que se acaba de describir.



(a) Iteración 3

(b) Iteración 4

Figura 9.7: Curva con geometria serpentín en rectangulo 1x1. (Rodrigo Arbós. 2026)

De la misma forma que se ha hecho al principio del capítulo, sde puede ejecutar el código de la sección A.3 y ver el transporte de masa para entender mejor la geometria.

https://youtu.be/KyhWu6Ewwjs

Figura 9.8: Transporte de masa por Serpentín con iteración 4. (Rodrigo Arbós. 2026)

10. Resultados

En este capítulo se presentan los resultados obtenidos a partir de las simulaciones. Con las funciones desarrolladas durante el trabajo, se analiza el comportamiento de la transferencia térmica en distintas iteraciones de las geometrías estudiadas.

Para las simulaciones, se hace fluir agua por el canal. Esto obliga a parametrizar las constantes de la siguiente manera:

```
T_in  = float(55.0)    # ( C ) temperatura de entrada del fluido
T_ext = float(17.0)    # ( C ) temperatura exterior
c_p = float(4186.0)    # J/(kg K) calor espec fico del fluido
d   = float(1000.0)    # kg/m  densidad del fluido
h = float(0.05)        # (m) altura de la tuber a (fija)
```

Las demás variables, a excepción de los coeficientes de transferencia térmica, vienen definidas por la discretización de la superficie o se calculan a partir de estas.

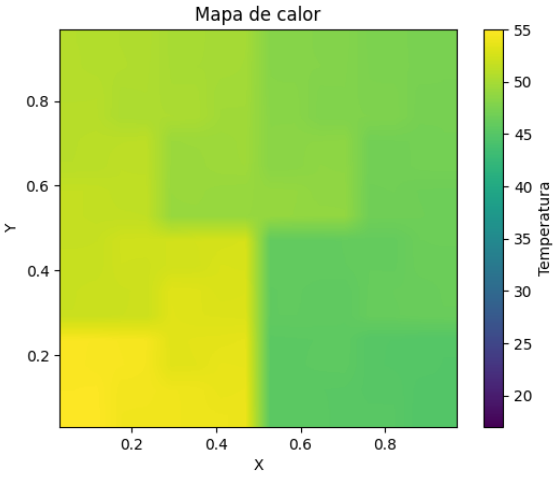
Si se ejecuta la primera simulación con 4 iteraciones en la codificación de las curvas, se obtienen los siguientes resultados:

$$\textit{Distancia entre nodos } Dx = 0{,}0625 \textit{ y } Dy = 0{,}0625$$

$$\textit{Discretización del tiempo } dt = 7{,}8125 \textit{ y caudal másico } \dot{\boldsymbol{m}} = 0{,}025$$

Convergencia en 548 *iteraciones*

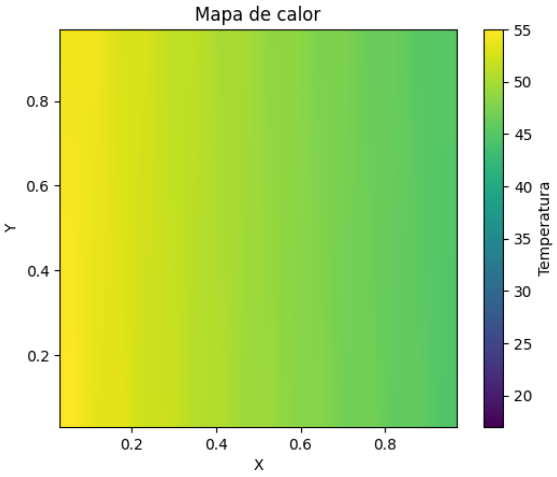
Temperatura mínima = 44,7629288



(a) Hilbert iteración 4

Convergencia en 385 *iteraciones*

Temperatura mínima = 44,7166404



(b) Serpentín iteración 4

Figura 10.1: Equilibrio térmico en rectángulo 1×1. (Rodrigo Arbós. 2026)

No se utilizará otro fluido, ya que en aplicaciones habituales, como el suelo radiante o la refrigeración de componentes, se emplea agua. En algunos casos, esta se modifica con glicol, un líquido altamente soluble en agua que actúa como anticongelante, pero que reduce ligeramente el valor del calor específico, c_p. Por tanto, el fluido con mejor capacidad de transferencia térmica utilizado en situaciones reales es el agua.

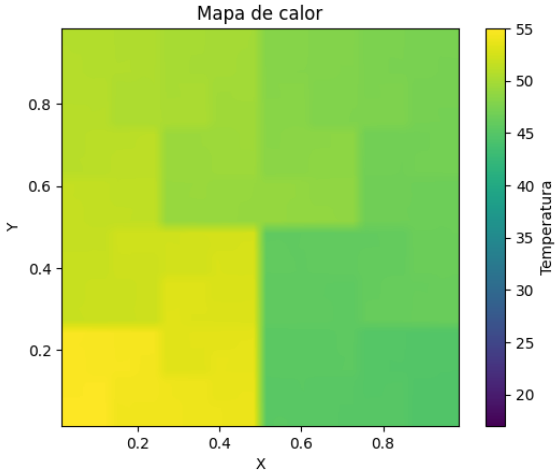
Si se ejecuta ahora la simulación con 5 iteraciones en el algoritmo de dibujar la curva se obtienen los siguientes resultados:

$$\textit{Distancia entre nodos } Dx = 0{,}03125 \textit{ y } Dy = 0{,}03125$$

$$\textit{Discretización del tiempo } dt = 1{,}953125 \textit{ y caudal másico } \dot{\boldsymbol{m}} = 0{,}025$$

Convergencia en 1953 *iteraciones*

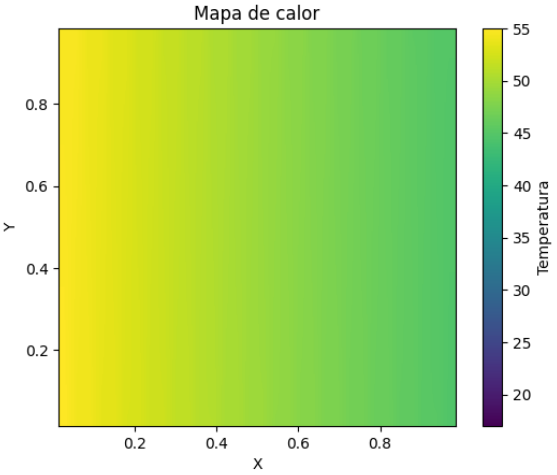
Temperatura mínima = 44,6134501



(a) Hilbert iteración 5

Convergencia en 1332 *iteraciones*

Temperatura mínima = 44,6657503



(b) Serpentín iteración 5

Figura 10.2: Equilibrio térmico en rectángulo 1×1. (Rodrigo Arbós. 2026)

Se puede suponer con un alto grado de certeza que la serpentina genera un gradiente unidimensional muy progresivo, mientras que la geometría de Hilbert escala el perfil en cuatro cuadrantes claramente diferenciados.

Para confirmar este resultado, se ejecuta el algoritmo con una iteración igual a 6 en la generación del recorrido. Como se observó en el ejemplo 7.13, la geometría prácticamente rellena todo el espacio disponible.

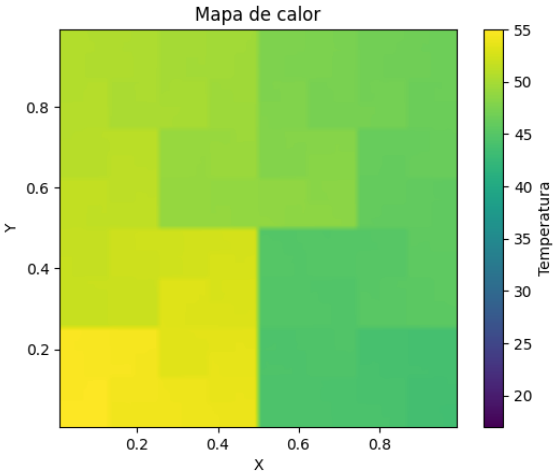
De esta manera, se obtiene una simulación computacionalmente costosa, ya que el número de puntos por recorrido es de 4096 en total, pero se modela una geometría que se asemeja en gran medida a un panel radiante, ya que prácticamente se ocupa toda la superficie.

Distancia entre nodos $Dx = 0{,}015625$ *y* $Dy = 0{,}015625$

Discretización del tiempo $dt = 0{,}48828125$ *y caudal másico* $\dot{\boldsymbol{m}} = 0{,}025$

Convergencia en 5604 *iteraciones*

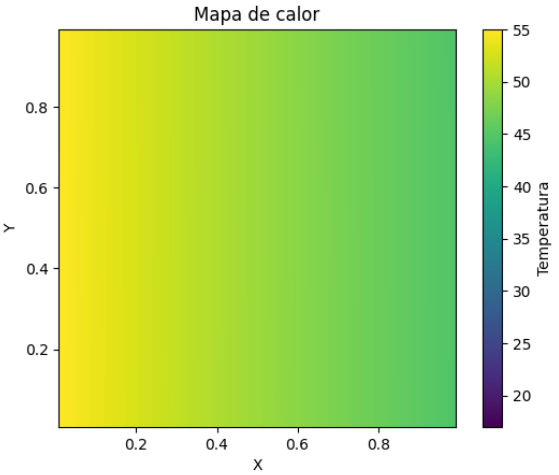
Temperatura mínima = 43,5441816



(a) Hilbert iteración 6

Convergencia en 4831 *iteraciones*

Temperatura mínima = 44,5881695



(b) Serpentín iteración 6

Figura 10.3: Equilibrio térmico en rectángulo 1×1. (Rodrigo Arbós. 2026)

Efectivamente, no se aprecia una diferencia gráfica en la superficie tras llegar al equilibrio térmico al aumentar el número de iteraciones del recorrido.
Por tanto, si se desea refrigerar de forma selectiva una placa con cuatro zonas bien diferenciadas, la geometría no convencional resulta adecuada para cubrir el plano, ya que permite separar cada región con diferentes comportamientos.

En los siguientes enlaces se puede encontrar la simulación completa del ejemplo 10.2 y ver cómo el panel alcanza el equilibrio térmico para la iteración 5 de las geometrías.

https://youtu.be/82j0rIPLza4

(a) Hilbert iteración 5

https://youtu.be/TwfwRn0etxM

(b) Serpentín iteración 5

Figura 10.4: Simulación de la transferencia térmica en rectángulo 1×1. (Rodrigo Arbós. 2026)

Si se analizan las simulaciones a nivel físico, se puede afirmar que, al aumentar el número de iteraciones, el canal se hace más fino, como se observa en los parámetros Dx y Dy. Este hecho, en una simulación real, provocaría un aumento de la fricción.
Para mantener el mismo caudal volumétrico de 1,5 L/min, que determina la constante $\dot{\mathbf{m}}$ (caudal másico), sería necesario emplear una bomba de mayor potencia para mover el fluido.

Por otro lado, cuando hay poca fricción en un canal con fluido, el líquido desarrolla una capa interna característica del **régimen laminar**, que actúa como un pequeño aislante térmico, dificultando la disipación de calor, fenómeno que no es posible considerar en estas simulaciones.

No obstante, se puede afirmar que, en la curva de Hilbert, debido a la presencia de múltiples giros de 90° en su geometría, se produce una mezcla interna del fluido de forma continua. Este efecto rompe de manera recurrente la capa aislante propia del régimen laminar, lo que podría mejorar notablemente la eficiencia de la transferencia térmica por convección, siempre que se disponga de la potencia necesaria para impulsar el fluido.

10.1. Conclusión global

La curva de Hilbert permite obtener una entropía mayor, ya que hace circular el fluido tanto en el eje x como en el eje y de forma equilibrada. En cambio, la curva serpenteante desplaza el fluido principalmente a lo largo del eje y durante la mayor parte de la simulación.

El concepto de **entropía** fue introducido por Rudolf Clausius en el siglo XIX en el marco del desarrollo de la termodinámica clásica (Clausius, 1865).
Es una magnitud física propia de los sistemas termodinámicos que mide el número de microestados compatibles con un determinado macroestado, es decir, cuantifica el grado de desorden del sistema al llegar al estado estacionario.
En términos generales, para procesos reversibles, la variación de entropía se expresa como el cociente entre el calor intercambiado y la temperatura (Çengel y Ghajar, 2020).

Dado que la curva de Hilbert presenta, en todas las simulaciones y para el mismo orden de la geometría del recorrido, un mayor número de iteraciones, se tiene un mayor número de intercambios de calor, por lo que se puede afirmar que el sistema alcanza un mayor nivel de entropía. Además, desde un punto de vista gráfico, se observa que la distribución

de temperaturas en el equilibrio térmico presenta una mayor variedad de microestados en comparación con la simulación serpenteante.

A pesar de que no se observa una mejora significativa en la transferencia térmica —debido a que las diferencias de temperatura finales son reducidas—, si se recuerdan las propiedades de la curva de Hilbert (1.1.1), esta presenta una notable preservación de la localidad. Este comportamiento se refleja en las imágenes finales, donde se aprecia cómo, incluso en el degradado de la función de representación, se mantienen los subcuadrados correspondientes a la discretización de la región.
Esta propiedad pone de manifiesto que la curva de Hilbert es capaz de mapear el espacio bidimensional de forma fiel.

Estos resultados abren nuevas líneas de trabajo para continuar desarrollando este tipo de simulaciones y explorar el potencial de esta geometría en las aplicaciones de disipación de calor.

11. Conclusiones y trabajo futuro

El objetivo principal del trabajo es analizar, mediante imágenes y simulaciones, la transferencia térmica de la curva de Hilbert frente a la geometría serpenteante sobre una superficie.

A lo largo de una investigación intensa y con múltiples representaciones de estas geometrías, siguiendo como guía para la evolución de este trabajo los siguientes puntos, se ha alcanzado una conclusión clara.

Objetivo 1 **Definir formalmente una curva que rellene el espacio.**

Con las investigaciones proporcionadas por los libros mencionados en la motivación del trabajo, como (Sánchez, 2023) o (Sagan, 1994), se escogió la curva de Hilbert como candidata por sus excelentes propiedades, que se exponen en la sección 1.1.1.

Seguidamente, se realiza un estudio profundo sobre el campo, como se observa en el capítulo 5, y en el capítulo 7 se formaliza matemáticamente la geometría para crear un algoritmo propio y eficiente, y poder dibujar, con tan solo unas simples condiciones, el recorrido. Estas condiciones se pueden encontrar en la figura 7.5.

Objetivo 2 **Desarrollar la teoría sobre transferencia térmica y establecer las leyes que rigen el intercambio de calor del fluido que circula por el canal.**

Se puede aprender todo lo necesario para formalizar cualquier intercambio de temperatura en procesos de transferencia térmica si se consulta el libro de (Çengel y Ghajar, 2020).

Como se ha redactado en el capítulo 6, todo intercambio de temperatura viene definido de forma general por la fórmula de calor sensible, pero, realmente, los mecanismos por los cuales el material pierde o gana calor son tres: conducción, convección y radiación.

En cada una de las siguientes secciones del capítulo, 6.1, 6.2 y 6.3, se consigue formalizar con detalle cada uno de los tres mecanismos y definir la razón por la cual el fluido pasa de su temperatura inicial a la del equilibrio térmico.

Destaca la implementación de figuras creadas manualmente, como 6.1, para que se comprenda más fácilmente cada uno de los mecanismos y se visualice el funcionamiento del intercambio de temperatura.

Objetivo 3 **Codificar, a partir de la definición formal, una curva que rellene el espacio de un rectángulo cualquiera.**

Siguiendo con todo lo que se ha desarrollado en el objetivo 1, se programa en lenguaje Python cada una de las condiciones y los cálculos para discretizar la superficie según el orden de la geometría a representar.

De esta manera se consigue codificar, como se puede encontrar en el apéndice A.1.2, la función que implementa todas estas leyes y devuelve la lista ordenada de puntos por los que pasará la curva.
También se implementa una función en A.1.3 que dibuja por dónde pasa el canal sobre la superficie y muestra la discretización del rectángulo global en partes iguales.

Destaca l'adaptación del código a superficies rectangulares no cuadradas, como puede observarse en los ejemplos 7.11 y 7.14.

Objetivo 4 **Desarrollar la recurrencia general de la temperatura del fluido que circula por el canal hasta alcanzar el equilibrio térmico.**

Siguiendo con lo logrado en el objetivo 2, es decir, el desarrollo de la teoría sobre la transferencia térmica, se juntan todos estos conceptos con los establecidos de la curva de Hilbert para formalizar el capítulo 8. En este capítulo, se consigue formalizar la razón con la que se realizan las simulaciones del objetivo principal.

Es la sección más importante del trabajo antes de los resultados; se comprende el proceso de transporte de masa, que es el método por el cual la temperatura avanza por el canal como si del fluido se tratara, además de los tres procesos por los cuales el material pierde o gana calor mientras circula por la tubería.

Como se observar en la figura 8.2, para facilitar los cálculos, se codifica el trabajo sobre un canal rectangular.

Destaca cada una de las figuras que presentan todos los mecanismos de transferencia térmica y ejemplifican cada término de la recurrencia creada en 17. Recurrencia que da constancia del logro de este objetivo: formalizar la variación de temperatura de cada punto de la curva en el cálculo computacional. Esta formulación muestra cómo el fluido consigue estabilizarse en el equilibrio térmico.

Objetivo 5 **Simular la recurrencia de temperatura mediante cálculo computacional hasta alcanzar el equilibrio térmico.**

Una vez definida la formulación matemática por la cual la tubería del problema evoluciona térmicamente, en los apéndices A.1.6 y A.1.8 se puede encontrar la implementación en Python de cada uno de los términos de la recurrencia.

En el capítulo 9.3 se muestra de forma gráfica el resultado de una simulación sin pérdida de calor, donde se observa cómo la superficie alcanza la misma temperatura con la que entra el fluido.

Destaca que se ha desarrollado una función, recogida en el apéndice A.3, que permite generar un vídeo del cálculo computacional de la recurrencia, lo que facilita la visualización de la evolución térmica de la superficie hasta alcanzar el equilibrio térmico, tal como se establece en este objetivo. De este modo, no solo se dispone de una imagen final, sino también de la transformación completa del sistema.

Objetivo 6 **Analizar la situación de equilibrio térmico y compararla con la trayectoria del serpentín convencional.**

Finalmente, con todo el código desarrollado y la información recopilada a lo largo del trabajo, se ha conseguido realizar una comparación objetiva de las distintas simulaciones, identificando las ventajas e inconvenientes de la geometría no convencional frente a la serpentina común.

En la parte final del capítulo 10 se puede encontrar un enlace para visualizar el vídeo de las simulaciones completas con pérdida de calor a través de todos los mecanismos considerados, lo que permite observar cómo la superficie alcanza el equilibrio térmico de forma progresiva y cómo se genera un gradiente térmico muy diferente en cada caso.

En conclusión, por todo lo expuesto, se puede afirmar que el objetivo principal del trabajo:

Analizar y comparar, mediante simulaciones, la transferencia térmica de la geometría de un canal basado en la curva de Hilbert frente al serpentín convencional sobre una superficie.

se ha alcanzado satisfactoriamente. El análisis realizado en el capítulo 10 ha permitido comparar de forma objetiva ambas configuraciones, evidenciando que el serpentín convencional tiene una mayor simplicidad constructiva, mientras que la geometría basada en la curva de Hilbert introduce un comportamiento térmico más complejo y distribuido.

Si bien los resultados obtenidos no muestran una mejora significativa en la transferencia térmica global, sí se observa que la geometría de Hilbert genera distribuciones de temperatura más heterogéneas, derivadas de la combinación continua de los ejes x e y en el recorrido del fluido.
Asimismo, este suceso, que va vinculado con fenómenos físicos no considerados en la simulación —como las turbulencias que se generan en el fluido o la rotura de la capa límite—, podría derivar en mejoras adicionales en la eficiencia térmica en condiciones reales.

Finalmente, teniendo en cuenta los avances actuales en técnicas de fabricación, como la impresión 3D y otros procesos de manufactura avanzada, la implementación de esta geometría compleja, la curva de Hilbert, deja de ser una limitación. Esto abre la posibilidad de explorar físicamente la configuración estudiada y demostrar la mejora en la transferencia térmica sobre superficies.
Aun así, se ha creado una base computacional muy importante que permite explotar al máximo esta herramienta a partir de las propiedades de la geometría. Esto abre puertas a imaginar productos que puedan sacarle mucho provecho a estas funcionalidades.

11.1. Trabajo futuro

En cuanto al TFM, la investigación podría ampliarse incorporando nuevas simulaciones con tecnología más avanzada. Como se ha visto en los artículos de (Liu y cols., 2025) y (Jaffal y cols., 2023), se implementan simulaciones CFD. Existen programas como OpenFOAM que permiten visualizar de forma más precisa que Python cómo se transfiere el calor en los procesos térmicos. En estas aplicaciones no es necesario definir cada uno de los términos del intercambio, sino que basta con indicar las condiciones iniciales, y el software muestra gráficamente, cómo el fluido circula por el canal y evoluciona térmicamente.

Asimismo, se podría avanzar en el estudio de la geometría, extendiendo el algoritmo a superficies no rectangulares, adaptándolo a configuraciones más realistas. En relación con la evaluación de los resultados, resultaría de interés realizar una comparación cuantitativa más detallada para proporcionar un análisis numérico más riguroso.

Finalmente, como se ha explicado en las conclusiones, se pueden explotar al máximo las propiedades de la nueva geometría, lo que abre la posibilidad de desarrollar un producto novedoso basado en esta tecnología. Asimismo, publicar en un futuro la investigación para aportar a la comunidad científica un avance significativo.

Referencias

Clausius, R. (1865). Ueber verschiedene für die anwendung bequeme formen der hauptgleichungen der mechanischen wärmetheorie. *Annalen der Physik und Chemie*, *125*, 353–400.

Hilbert, D. (1891). Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, *38*, 459–460.

Incropera, F. P., DeWitt, D. P., Bergman, T. L., y Lavine, A. S. (2007). *Fundamentals of heat and mass transfer* (6.ª ed.). New York: John Wiley & Sons.

Jaffal, H. M., Mahmoud, N. S., Imran, A. A., y Hasan, A. (2023). Performance enhancement of a novel serpentine channel cooled plate used for cooling of li-ion battery module. *International Journal of Thermal Sciences*.

Liu, J., Sun, J., Yin, C., Wang, C., y Sun, J. (2025). Revealing the heat transfer enhancement mechanism of dimpled tube hybridized with twisted tape via dimple utility indicator. *Case Studies in Thermal Engineering*.

Mandelbrot, B. B. (1982). *The fractal geometry of nature*. W. H. Freeman.

Peano, G. (1890). Sur une courbe, qui remplit toute une aire plane. *Mathematische Annalen*, *36*, 157–160. doi: 10.1007/BF01444019

Radzai, M. H. M., Yaw, C. T., Lim, C. W., Koh, S. P., y Ahmad, N. A. (2021). Numerical analysis on the performance of a radiant cooling panel with serpentine-based design. *Energies*.

Sagan, H. (1994). *Space-filling curves*. Springer.

Sánchez, Ó. (2023). *Fractales y caos en ingeniería y aplicaciones*. Universidad Nacional de Colombia.

Yuan, Y., Cao, J., Zhang, Z., Xiao, Z., y Wang, X. (2024). Experimental and numerical simulation study of a novel double shell-passes multi-layer helically coiled tubes heat exchanger. *International Journal of Heat and Mass Transfer*.

Çengel, Y. A., y Ghajar, A. J. (2020). *Transferencia de calor y masa: fundamentos y aplicaciones*. McGraw-Hill.

A. Apéndice

A.1. Código para simulación

En esta sección se encuentran las celdas de código Python diferenciadas por sus funciones que permiten simular todos los resultados que se han visto durante el trabajo.

A.1.1. Librerías

```
import matplotlib.pyplot as plt
import numpy as np
```

A.1.2. Recorrido de la tubería

La siguiente celda implementa una función que devuelve los puntos por los que pasa la curva de Hilbert en un rectangulo de tamaño cualquiera.

```
def centros_hilbert(x_max, y_max, iteraciones):

    Genera 4^iteraciones centros de sub-rectangulos que forman
        vertices de curva Hilbert.

    x_max, y_max : tama o del rect ngulo original (desde (0,0))
    iteraciones  : numero de subdivisiones del rectangulo


    # Empezamos con 1 centro: el centro del rectangulo grande
    centros = [(x_max / 2, y_max / 2)]

    # Medidas iniciales del rectangulo
    w = x_max
    h = y_max

    #1a iteracion del algoritmo de Hilbert
    # reducimos a la mitad
    w /= 2
    h /= 2
```

```
    # Offset desde el centro hacia los 4 nuevos centros
    dx = w / 2
    dy = h / 2
    nuevos = []
    (cx,cy) = centros[0]
    nuevos.extend([
                (cx - dx, cy - dy),   # inferior izquierdo
                (cx - dx, cy + dy),   # superior izquierdo
                (cx + dx, cy + dy),   # superior derecho
                (cx + dx, cy - dy),   # inferior derecho
            ])
    centros = nuevos

    for j in range(iteraciones-1):
        nuevos = []
        # Al subdividir cada rectangulo, su tama o se reduce a la mitad
        w /= 2
        h /= 2
        # Offset desde el centro hacia los 4 nuevos centros
        dx = w / 2
        dy = h / 2

        for i in range(0, len(centros), 4):
            (cx1, cy1), (cx2, cy2), (cx3, cy3), (cx4, cy4) = centros[i:i+4]
            if cx1 == cx2:
                if cx1 < cx3:
                    nuevos.extend([
                        (cx1 - dx, cy1 - dy),   # inferior izquierdo
                        (cx1 + dx, cy1 - dy),   # inferior derecho
                        (cx1 + dx, cy1 + dy),   # superior derecho
                        (cx1 - dx, cy1 + dy),   # superior izquierdo
                    ])
                    nuevos.extend([
```

```
                (cx2 - dx, cy2 - dy),   # inferior izquierdo
                (cx2 - dx, cy2 + dy),   # superior izquierdo
                (cx2 + dx, cy2 + dy),   # superior derecho
                (cx2 + dx, cy2 - dy),   # inferior derecho
            ])
            nuevos.extend([
                (cx3 - dx, cy3 - dy),   # inferior izquierdo
                (cx3 - dx, cy3 + dy),   # superior izquierdo
                (cx3 + dx, cy3 + dy),   # superior derecho
                (cx3 + dx, cy3 - dy),   # inferior derecho
            ])
            nuevos.extend([
                (cx4 + dx, cy4 + dy),   # superior derecho
                (cx4 - dx, cy4 + dy),   # superior izquierdo
                (cx4 - dx, cy4 - dy),   # inferior izquierdo
                (cx4 + dx, cy4 - dy),   # inferior derecho
            ])
        else:
            nuevos.extend([
                (cx1 + dx, cy1 + dy),   # superior derecho
                (cx1 - dx, cy1 + dy),   # superior izquierdo
                (cx1 - dx, cy1 - dy),   # inferior izquierdo
                (cx1 + dx, cy1 - dy),   # inferior derecho
            ])
            nuevos.extend([
                (cx2 + dx, cy2 + dy),   # superior derecho
                (cx2 + dx, cy2 - dy),   # inferior derecho
                (cx2 - dx, cy2 - dy),   # inferior izquierdo
```

```
                        (cx2 - dx, cy2 + dy),   # superior izquierdo
                    ])
                    nuevos.extend([
                        (cx3 + dx, cy3 + dy),   # superior derecho
                        (cx3 + dx, cy3 - dy),   # inferior derecho
                        (cx3 - dx, cy3 - dy),   # inferior izquierdo
                        (cx3 - dx, cy3 + dy),   # superior izquierdo
                    ])
                    nuevos.extend([
                        (cx4 - dx, cy4 - dy),   # inferior izquierdo
                        (cx4 + dx, cy4 - dy),   # inferior derecho
                        (cx4 + dx, cy4 + dy),   # superior derecho
                        (cx4 - dx, cy4 + dy),   # superior izquierdo
                    ])
            else:
                if cx1 < cx3:
                    nuevos.extend([
                        (cx1 - dx, cy1 - dy),   # inferior izquierdo
                        (cx1 - dx, cy1 + dy),   # superior izquierdo
                        (cx1 + dx, cy1 + dy),   # superior derecho
                        (cx1 + dx, cy1 - dy),   # inferior derecho
                    ])
                    nuevos.extend([
                        (cx2 - dx, cy2 - dy),   # inferior izquierdo
                        (cx2 + dx, cy2 - dy),   # inferior derecho
                        (cx2 + dx, cy2 + dy),   # superior derecho
                        (cx2 - dx, cy2 + dy),   # superior izquierdo
                    ])
```

```
                    nuevos.extend([
                        (cx3 - dx, cy3 - dy),   # inferior izquierdo
                        (cx3 + dx, cy3 - dy),   # inferior derecho
                        (cx3 + dx, cy3 + dy),   # superior derecho
                        (cx3 - dx, cy3 + dy),   # superior izquierdo
                    ])
                    nuevos.extend([
                        (cx4 + dx, cy4 + dy),   # superior derecho
                        (cx4 + dx, cy4 - dy),   # inferior derecho
                        (cx4 - dx, cy4 - dy),   # inferior izquierdo
                        (cx4 - dx, cy4 + dy),   # superior izquierdo
                    ])
                else:
                    nuevos.extend([
                        (cx1 + dx, cy1 + dy),   # superior derecho
                        (cx1 + dx, cy1 - dy),   # inferior derecho
                        (cx1 - dx, cy1 - dy),   # inferior izquierdo
                        (cx1 - dx, cy1 + dy),   # superior izquierdo
                    ])
                    nuevos.extend([
                        (cx2 + dx, cy2 + dy),   # superior derecho
                        (cx2 - dx, cy2 + dy),   # superior izquierdo
                        (cx2 - dx, cy2 - dy),   # inferior izquierdo
                        (cx2 + dx, cy2 - dy),   # inferior derecho
                    ])
                    nuevos.extend([
                        (cx3 + dx, cy3 + dy),   # superior derecho
                        (cx3 - dx, cy3 + dy),   # superior izquierdo
```

```
                        (cx3 - dx, cy3 - dy),    # inferior izquierdo
                        (cx3 + dx, cy3 - dy),    # inferior derecho
                    ])
                    nuevos.extend([
                        (cx4 - dx, cy4 - dy),    # inferior izquierdo
                        (cx4 - dx, cy4 + dy),    # superior izquierdo
                        (cx4 + dx, cy4 + dy),    # superior derecho
                        (cx4 + dx, cy4 - dy),    # inferior derecho
                    ])

        centros = nuevos  # Actualizamos la lista completa

    return (centros, 2*dx, 2*dy)
```

A.1.3. Dibujar la curva

En la siguiente celda se implementa una función que representa el rectangulo de la superficie y la curva de Hilbert sobre ella.

```
def dibujar_tuberia(centros):
  # Crear figura
  plt.figure(figsize=(10,10))

  # Dibujar centros
  xs = [c[0] for c in centros]
  ys = [c[1] for c in centros]
  plt.scatter(xs, ys, marker='o', s=(9-iteraciones)**2)
  plt.plot(xs,ys)

  # Dibujar l neas de subdivisi n
  step_x = x_max / (2**iteraciones)
  step_y = y_max / (2**iteraciones)

  # L neas verticales
  for i in range(1, 2**iteraciones):
```

```
        x = i * step_x
        plt.plot([x, x], [0, y_max], color='gray', linestyle='--')

    # L neas horizontales
    for i in range(1, 2**iteraciones):
        y = i * step_y
        plt.plot([0, x_max], [y, y], color='gray', linestyle='--')

    # Configuraci n del plano
    plt.xlim(0, x_max)
    plt.ylim(0, y_max)
    plt.gca().set_aspect('equal', 'box')
    plt.xlabel( X )
    plt.ylabel( Y )
    plt.title( CURVA DE HILBERT )

    plt.show()
```

A.1.4. Crear tubería con temperatura

En las siguientes líneas de código se presenta la función que recibe una lista ordenada de puntos, la cual define el recorrido del canal. A partir de esta información, la función devuelve un diccionario que permite representar la temperatura asociada a cada nodo de la tubería sobre la superficie.

```
def crear_tuberia(centros, temperatura):
    return {punto: temperatura for punto in centros}
```

A.1.5. Representar la temperatura sobre la superficie

En la siguiente celda de código se encuentran dos funciones que dibujan como se distribuye la temperatura por la superficie al circular por el canal. La mínima coge un color frio y la màxima un color càlido. La primera función muestra los nodos por separado, en cambio, la segunda función representa todo el cuadrado pintado. En esta segunda se pueden seleccionar dos formas distintas de pintar la temperatura:

1. *interpolation* = *bilinear* : Crea un degradado entre cada subcuadrado.

2. *aspect = equal* : Representa cada cuadrado todo pintado de la temperatura correspondiente, sin degradar y combinar las temperaturas de los nodos adyacentes.

```
def plot_heatmap_scatter(temp_dict):
    xs = [p[0] for p in temp_dict]
    ys = [p[1] for p in temp_dict]
    ts = [temp_dict[p] for p in temp_dict]

    plt.figure()
    sc = plt.scatter(xs, ys, c=ts)
    plt.colorbar(sc, label= Temperatura )
    plt.xlabel( X )
    plt.ylabel( Y )
    plt.title( Mapa de calor (scatter) )
    plt.axis( equal )
    plt.show()

def plot_heatmap_grid(temp_dict,T_min,T_max):
    xs = sorted(set(x for x, _ in temp_dict))
    ys = sorted(set(y for _, y in temp_dict))

    x_index = {x: i for i, x in enumerate(xs)}
    y_index = {y: i for i, y in enumerate(ys)}

    grid = np.full((len(ys), len(xs)), np.nan)

    for (x, y), T in temp_dict.items():
        grid[y_index[y], x_index[x]] = T

    plt.figure()
    plt.imshow(
        grid,
        origin= lower ,
        extent=[min(xs), max(xs), min(ys), max(ys)],
        interpolation= bilinear , #(hay degradado)
        vmin=T_min,
        vmax=T_max     # Escala fija
        #aspect =  equal  #(no hay degradado)
```

```

    )
    plt.colorbar(label= Temperatura )
    plt.xlabel( X )
    plt.ylabel( Y )
    plt.title( Mapa de calor )
    plt.show()
```

A.1.6. Términos de la recurrencia

En esta sección se codifican las funciones que corresponden a cada término de la recurrencia principal. Cada una devuelve el calor que intercambia cada nodo por medio del mecanismo que codifican.

1. **Convección interna**

```
def conveccion_int(t_i, t_ant, h_int, A_lat):
  return h_int * A_lat * (t_ant - t_i)
```

2. **Convección externa**

```
def conveccion_ext(t_i, t_ext, h_ext, A_sup):
  return h_ext * A_sup * (t_ext - t_i)
```

3. **Conducción**

```
def conduccion(t_j, t_i, k, A_lat):
  return k * A_lat * (t_j - t_i)
```

4. **Radiación**

```
def radiacion(t_i, t_ext, q, A_sup):
  sigma = 5.670374419e-8 # Stefann-Boltzman
  Ti = t_i + 273.15
  Text = t_ext + 273.15
  return q * sigma * A_sup * (Ti**4 - Text**4)
```

A.1.7. Encontrar vecinos no adyacentes

Para aplicar la conducción se necesita tener una lista de los vecinos adyacentes dentro de la superficie pero que estos no sean consecutivos en el camino de la tubería. Para ello se ha creado la siguiente función que devuelve un conjunto de todos estos puntos dado el índice del nodo actual.

```
def vecinos(i, centros, x_max, y_max, A_x , A_y):
  nodo = centros[i]
  vecinos = [(nodo[0]+A_x,nodo[1]), (nodo[0]-A_x,nodo[1]), (nodo
      [0],nodo[1]+A_y), (nodo[0],nodo[1]-A_y)]
  evitar = (centros[(i+1)%(len(centros))], centros[i-1])
  vecinos = [x for x in vecinos if x not in evitar]
  vecinos = [ (x, y) for (x, y) in vecinos if 0 < x < x_max and 0 <
      y < y_max]

  return vecinos
```

A.1.8. Algoritmo principal

En la siguiente celda esta codificada la recurrencia principal del trabajo. Dadas las condiciones iniciales y la lista de puntos ordenada por donde circula la tubería, crea la tubería de temperaturas y itera los puntos siguiendo la reucerrencia de la teoria explicada en la sección 17 para llegar al equilibrio térmico. Como resultado devuelve un diccionario que representa la temperatura del canal una vez llegado al equilibrio térmico.

```
def equilibrio_termico(centros, Dx, Dy, m, c_p, k, q, h_int, h_ext,
    A_f, A_lat, A_sup, x_max, y_max, T_in, T_ext, max_iter):
  eps = 1e-3
  tub = crear_tuberia(centros, T_ext)
  tub[centros[0]] = T_in

  # VELOCIDAD A LA QUE AVANZA EL FLUIDO
  v = 2.5e-5 # caudal volum trico constante
  dt = (A_f * Dx) / v

  print(f'discretizaci n del tiempo dt = {dt}')
  print(f'obtenemos caudal m sico m   = {m/dt}')
```

```
12
13  next_tub = tub.copy()
14  for n in range(max_iter):
15      max_change = 0.0
16
17      for i in range(0, len(centros)):
18        p_i = centros[i]
19
20        T_i = tub[p_i]
21
22        # 1.1) Conveccion interna next
23        if i == len(centros)-1:
24          term_conv_int_next = 0
25        else:
26          p_next = centros[i+1]
27          T_next = tub[p_next]
28          term_conv_int_next = conveccion_int(T_i, T_next, h_int,
              A_lat)
29        # 1.2) Conveccion interna ant
30        if i == 0:
31          term_conv_int_ant = 0
32        else:
33          p_ant = centros[i - 1]
34          T_ant = tub[p_ant]
35          term_conv_int_ant = conveccion_int(T_i, T_ant, h_int,
              A_lat)
36
37        # 2) Conveccion externa
38        term_conv_ext = conveccion_ext(T_i, T_ext, h_ext, A_sup)
39
40        # 2) Conduccion : suma vecinos existentes
41        term_cond = 0.0
42        for p_j in vecinos(i, centros, x_max, y_max, Dx, Dy):
43              T_j = tub[p_j]
44              term_cond += conduccion(T_j, T_i, k, A_lat)
45
46        # 3) Radiacion : se resta
```

```
        term_rad = radiacion(T_i, T_ext, q, A_sup)

        # Actualizacion
        Ti_new = T_i + dt * (term_conv_int_next + term_conv_int_ant
            + 2*term_conv_ext + term_cond - 2*term_rad)/(m*c_p)
        next_tub[p_i] = Ti_new

      # Avanzamos el flu do
      for i in range(len(centros)-1, 0, -1):   # va de N-1      1
        next_tub[centros[i]] = next_tub[centros[i-1]]
        max_change = max(max_change, next_tub[centros[i]]-tub[
            centros[i]])
      next_tub[centros[0]] = T_in

      # criterio de equilibrio
      if max_change < eps:
        print(f CONVERGENCIA EN {n} ITERACIONES )
        return next_tub

      plot_heatmap_grid(next_tub,T_ext,T_in)  # PARA VER LA
          EVOLUCI N
      tub = next_tub.copy()

  print('NO HAY CONVERGENCIA\n')
  return tub
```

A.2. Tubería serpentín

En esta sección se encuentra el código para poder ejecutar la transferencia térmica sobre la geometria común de serpentín. Se codifica solo la función que crea el recorrido de la tubería porque para la simulación se pueden aplicar las anteriores funciones.

```
def centros_serpentina(x_max, y_max, iteraciones):
  dx = x_max / 2**(iteraciones+1)
  dy = y_max / 2**(iteraciones+1)

  centros = []
```

```python

  for i in range(2**iteraciones):

    if(i%2 == 0):
      # subimos
      (x,y) = (dx + dx*(2*i),dy)
      centros.append((x,y))
      while (y < y_max-dy):
        (x,y) = (x,y+2*dy)
        centros.append((x,y))

    else:
      # bajamos
      (x,y) = (dx + dx*(2*i),y_max - dy)
      centros.append((x,y))
      while (y > dy):
        (x,y) = (x,y-2*dy)
        centros.append((x,y))

  return (centros, 2*dx, 2*dy)
```

A.3. Generar video de simulación

En las siguientes celdas se presentan los códigos necesarios para ejecutar el algoritmo que simula el comportamiento del sistema. A diferencia de la implementación principal, en cada iteración se genera una imagen que posteriormente es almacenada.

Estas imágenes se combinan para crear un vídeo que permite visualizar de forma detallada la evolución del proceso de transferencia térmica sobre la superficie.

```python
import os

def save_heatmap_frame(temp_dict, T_min, T_max, out_dir, frame_idx,
    dpi=150):
    os.makedirs(out_dir, exist_ok=True)

    xs = sorted(set(x for x, _ in temp_dict))
    ys = sorted(set(y for _, y in temp_dict))
```

```

    x_index = {x: i for i, x in enumerate(xs)}
    y_index = {y: i for i, y in enumerate(ys)}

    grid = np.full((len(ys), len(xs)), np.nan)
    for (x, y), T in temp_dict.items():
        grid[y_index[y], x_index[x]] = T

    fig, ax = plt.subplots()
    im = ax.imshow(
        grid,
        origin= lower ,
        extent=[min(xs), max(xs), min(ys), max(ys)],
        interpolation= bilinear ,
        vmin=T_min,
        vmax=T_max
    )
    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label( Temperatura )

    ax.set_xlabel( X )
    ax.set_ylabel( Y )
    ax.set_title(f Mapa de calor (iteraci n = {frame_idx}) )

    frame_path = os.path.join(out_dir, f frame_{frame_idx:06d}.png
        )
    fig.tight_layout()
    fig.savefig(frame_path, dpi=dpi)
    plt.close(fig)   # IMPORTANT SIMO para no acumular memoria

    return frame_path
```

```
import cv2
import glob

def frames_to_video(frames_dir, out_video_path, fps=20):
    frame_paths = sorted(glob.glob(os.path.join(frames_dir,  frame_
```

```
        *.png )))
    if not frame_paths:
        raise RuntimeError(f No se encontraron frames en {
            frames_dir} )

    first = cv2.imread(frame_paths[0])
    if first is None:
        raise RuntimeError(f No se pudo leer: {frame_paths[0]} )
    h, w, _ = first.shape

    fourcc = cv2.VideoWriter_fourcc(* mp4v )  # mp4 compatible
    writer = cv2.VideoWriter(out_video_path, fourcc, fps, (w, h))

    for p in frame_paths:
        img = cv2.imread(p)
        if img is None:
            raise RuntimeError(f No se pudo leer: {p} )
        if img.shape[1] != w or img.shape[0] != h:
            img = cv2.resize(img, (w, h), interpolation=cv2.
                INTER_AREA)
        writer.write(img)

    writer.release()
    return out_video_path
```

```
def video_equilibrio_termico(centros, Dx, Dy, m, c_p, k, q, h_int,
    h_ext, A_f, A_lat, A_sup, x_max, y_max, T_in, T_ext, max_iter):
  eps = 1e-3
  tub = crear_tuberia(centros, T_ext)
  tub[centros[0]] = T_in

  # VELOCIDAD A LA QUE AVANZA EL FLUIDO
  v = 2.5e-5 # caudal volum trico constante
  dt = (A_f * Dx) / v

  print(f'discretizaci n del tiempo dt = {dt}')
  print(f'obtenemos caudal m sico m   = {m/dt}')
```

```python

frames_dir =  frames
frame_idx = 0
# Guardar frame (por ejemplo cada iteraci n)
save_heatmap_frame(tub, T_ext, T_in, frames_dir, frame_idx)
frame_idx += 1
print(f'SITUACION INICIAL')
plot_heatmap_grid(tub, T_ext, T_in)

next_tub = tub.copy()
for n in range(max_iter):
    max_change = 0.0

    for i in range(0, len(centros)):
      p_i = centros[i]

      T_i = tub[p_i]

      # 1.1) Conveccion interna next
      if i == len(centros)-1:
        term_conv_int_next = 0
      else:
        p_next = centros[i+1]
        T_next = tub[p_next]
        term_conv_int_next = conveccion_int(T_i, T_next, h_int, A_lat)
      # 1.2) Conveccion interna ant
      if i == 0:
        term_conv_int_ant = 0
      else:
        p_ant = centros[i - 1]
        T_ant = tub[p_ant]
        term_conv_int_ant = conveccion_int(T_i, T_ant, h_int, A_lat)

      # 2) Conveccion externa
      term_conv_ext = conveccion_ext(T_i, T_ext, h_ext, A_sup)
```

```
47
48          # 2) Conduccion : suma vecinos existentes
49          term_cond = 0.0
50          for p_j in vecinos(i, centros, x_max, y_max, Dx, Dy):
51                T_j = tub[p_j]
52                term_cond += conduccion(T_j, T_i, k, A_lat)
53
54          # 3) Radiacion : se resta
55          term_rad = radiacion(T_i, T_ext, q, A_sup)
56
57          # Actualizacion
58          Ti_new = T_i + dt * (term_conv_int_next + term_conv_int_ant
                + 2*term_conv_ext + term_cond - 2*term_rad)/(m*c_p)
59          next_tub[p_i] = Ti_new
60
61        # Avanzamos el flu do
62        for i in range(len(centros)-1, 0, -1):   # va de N-1     1
63          next_tub[centros[i]] = next_tub[centros[i-1]]
64          max_change = max(max_change, next_tub[centros[i]]-tub[
               centros[i]])
65        next_tub[centros[0]] = T_in
66
67        # criterio de equilibrio
68        if max_change < eps:
69          print(f CONVERGENCIA EN {n} ITERACIONES )
70          save_heatmap_frame(next_tub, T_ext, T_in, frames_dir,
               frame_idx)
71          break
72
73        # Guardar frame (por ejemplo cada iteraci n)
74        save_heatmap_frame(next_tub, T_ext, T_in, frames_dir,
             frame_idx)
75        frame_idx += 1
76        tub = next_tub.copy()
77
78    print('NO HAY CONVERGENCIA\n')
79    # Al final, crear v deo
```

```
    video_path = frames_to_video(frames_dir,  simulacion.mp4 , fps
        =20)
    print( V deo guardado en: , video_path)
    return tub
```

A.4. Codificar paredes de la tubería

Existen herramientas capaces de representar superficies tridimensionales a partir de su discretización en puntos en el espacio. En este contexto, como se ha visto en la sección del estado del arte, el problema abordado podría simularse mediante programas CFD.

Para ello, resulta más conveniente definir las paredes del canal en lugar del recorrido de la tubería sobre la superficie. La siguiente función implementa este proceso: a partir de un porcentaje que representa la proporción del vóxel ocupada por la tubería, se generan las paredes del canal por el que discurre la trayectoria de Hilbert sobre la superficie.

```
def tuberia_hilbert(x_max, y_max, iteraciones, porcentaje):

    Genera la tuberia de Hilbert por la que circula el fluido.
    x_max, y_max : tamano del rectangulo original (desde (0,0))
    iteraciones  : numero de subdivisiones del rectangulo
    porcentaje : grosor respecto el tamano del subcuadrado valor
        entre (0,1)

    # Empezamos con 1 centro: el centro del rectangulo grande
    centros = [(x_max / 2, y_max / 2)]
    # Medidas iniciales del rectangulo
    w = x_max
    h = y_max
    #1a iteracion del algoritmo de Hilbert
    # reducimos a la mitad
    w /= 2
    h /= 2
    # Offset desde el centro hacia los 4 nuevos centros
    dx = w / 2
    dy = h / 2
    nuevos = []
    (cx,cy) = centros[0]
```

```
    nuevos.extend([
                (cx - dx, cy - dy),
                (cx - dx, cy + dy),
                (cx + dx, cy + dy),
                (cx + dx, cy - dy),
            ])
    centros = nuevos
    for j in range(iteraciones-1):
        nuevos = []
        tuberia1 = []
        tuberia2 = []
        # se reduce a la mitad
        w /= 2
        h /= 2
        # Offset desde el centro hacia los 4 nuevos centros
        dx = w / 2
        dy = h / 2
        #ancho tuberia
        radio = min(dx,dy)-porcentaje*min(dx,dy)
        for i in range(0, len(centros), 4):
            (cx1, cy1), (cx2, cy2), (cx3, cy3), (cx4, cy4) =
                centros[i:i+4]
            if cx1 == cx2:
                if cx1 < cx3:
                    nuevos.extend([
                        (cx1 - dx, cy1 - dy),
                        (cx1 + dx, cy1 - dy),
                        (cx1 + dx, cy1 + dy),
                        (cx1 - dx, cy1 + dy),
                    ])
                    nuevos.extend([
                        (cx2 - dx, cy2 - dy),
                        (cx2 - dx, cy2 + dy),
                        (cx2 + dx, cy2 + dy),
                        (cx2 + dx, cy2 - dy),
                    ])
                    nuevos.extend([
```

```
                    (cx3 - dx, cy3 - dy),
                    (cx3 - dx, cy3 + dy),
                    (cx3 + dx, cy3 + dy),
                    (cx3 + dx, cy3 - dy),
                ])
                nuevos.extend([
                    (cx4 + dx, cy4 + dy),
                    (cx4 - dx, cy4 + dy),
                    (cx4 - dx, cy4 - dy),
                    (cx4 + dx, cy4 - dy),
                ])
                if j == iteraciones-2:
                    tuberia1.extend([
                        (cx1 - dx-radio, cy1 - dy+radio),
                        (cx1 + dx-radio, cy1 - dy+radio),
                        (cx1 + dx-radio, cy1 + dy-radio),
                        (cx1 - dx-radio, cy1 + dy-radio),
                    ])
                    tuberia1.extend([
                        (cx2 - dx-radio, cy2 - dy-radio),
                        (cx2 - dx-radio, cy2 + dy+radio),
                        (cx2 + dx+radio, cy2 + dy+radio),
                        (cx2 + dx+radio, cy2 - dy+radio),
                    ])
                    tuberia1.extend([
                        (cx3 - dx-radio, cy3 - dy+radio),
                        (cx3 - dx-radio, cy3 + dy+radio),
                        (cx3 + dx+radio, cy3 + dy+radio),
                        (cx3 + dx+radio, cy3 - dy-radio),
                    ])
                    tuberia1.extend([
                        (cx4 + dx+radio, cy4 + dy-radio),
                        (cx4 - dx+radio, cy4 + dy-radio),
                        (cx4 - dx+radio, cy4 - dy+radio),
                        (cx4 + dx+radio, cy4 - dy+radio),
                    ])
                    tuberia2.extend([
```

```
                        (cx1 - dx+radio, cy1 - dy-radio),
                        (cx1 + dx+radio, cy1 - dy-radio),
                        (cx1 + dx+radio, cy1 + dy+radio),
                        (cx1 - dx+radio, cy1 + dy+radio),
                    ])
                    tuberia2.extend([
                        (cx2 - dx+radio, cy2 - dy-radio),
                        (cx2 - dx+radio, cy2 + dy-radio),
                        (cx2 + dx-radio, cy2 + dy-radio),
                        (cx2 + dx-radio, cy2 - dy-radio),
                    ])
                    tuberia2.extend([
                        (cx3 - dx+radio, cy3 - dy-radio),
                        (cx3 - dx+radio, cy3 + dy-radio),
                        (cx3 + dx-radio, cy3 + dy-radio),
                        (cx3 + dx-radio, cy3 - dy-radio),
                    ])
                    tuberia2.extend([
                        (cx4 + dx-radio, cy4 + dy+radio),
                        (cx4 - dx-radio, cy4 + dy+radio),
                        (cx4 - dx-radio, cy4 - dy-radio),
                        (cx4 + dx-radio, cy4 - dy-radio),
                    ])
            else:
                nuevos.extend([
                    (cx1 + dx, cy1 + dy),
                    (cx1 - dx, cy1 + dy),
                    (cx1 - dx, cy1 - dy),
                    (cx1 + dx, cy1 - dy),
                ])
                nuevos.extend([
                    (cx2 + dx, cy2 + dy),
                    (cx2 + dx, cy2 - dy),
                    (cx2 - dx, cy2 - dy),
                    (cx2 - dx, cy2 + dy),
                ])
                nuevos.extend([
```

```
                    (cx3 + dx, cy3 + dy),
                    (cx3 + dx, cy3 - dy),
                    (cx3 - dx, cy3 - dy),
                    (cx3 - dx, cy3 + dy),
                ])
                nuevos.extend([
                    (cx4 - dx, cy4 - dy),
                    (cx4 + dx, cy4 - dy),
                    (cx4 + dx, cy4 + dy),
                    (cx4 - dx, cy4 + dy),
                ])
                if j == iteraciones-2:
                    tuberia1.extend([
                        (cx1 + dx+radio, cy1 + dy-radio),
                        (cx1 - dx+radio, cy1 + dy-radio),
                        (cx1 - dx+radio, cy1 - dy+radio),
                        (cx1 + dx+radio, cy1 - dy+radio),
                    ])
                    tuberia1.extend([
                        (cx2 + dx+radio, cy2 + dy+radio),
                        (cx2 + dx+radio, cy2 - dy-radio),
                        (cx2 - dx-radio, cy2 - dy-radio),
                        (cx2 - dx-radio, cy2 + dy-radio),
                    ])
                    tuberia1.extend([
                        (cx3 + dx+radio, cy3 + dy-radio),
                        (cx3 + dx+radio, cy3 - dy-radio),
                        (cx3 - dx-radio, cy3 - dy-radio),
                        (cx3 - dx-radio, cy3 + dy+radio),
                    ])
                    tuberia1.extend([
                        (cx4 - dx-radio, cy4 - dy+radio),
                        (cx4 + dx-radio, cy4 - dy+radio),
                        (cx4 + dx-radio, cy4 + dy-radio),
                        (cx4 - dx-radio, cy4 + dy-radio),
                    ])
                    tuberia2.extend([
```

```
                        (cx1 + dx-radio, cy1 + dy+radio),
                        (cx1 - dx-radio, cy1 + dy+radio),
                        (cx1 - dx-radio, cy1 - dy-radio),
                        (cx1 + dx-radio, cy1 - dy-radio),
                    ])
                    tuberia2.extend([
                        (cx2 + dx-radio, cy2 + dy-radio),
                        (cx2 + dx-radio, cy2 - dy+radio),
                        (cx2 - dx+radio, cy2 - dy+radio),
                        (cx2 - dx+radio, cy2 + dy+radio),
                    ])
                    tuberia2.extend([
                        (cx3 + dx-radio, cy3 + dy+radio),
                        (cx3 + dx-radio, cy3 - dy+radio),
                        (cx3 - dx+radio, cy3 - dy+radio),
                        (cx3 - dx+radio, cy3 + dy-radio),
                    ])
                    tuberia2.extend([
                        (cx4 - dx+radio, cy4 - dy-radio),
                        (cx4 + dx+radio, cy4 - dy-radio),
                        (cx4 + dx+radio, cy4 + dy+radio),
                        (cx4 - dx+radio, cy4 + dy+radio),
                    ])
            else:
                if cx1 < cx3:
                    nuevos.extend([
                        (cx1 - dx, cy1 - dy),
                        (cx1 - dx, cy1 + dy),
                        (cx1 + dx, cy1 + dy),
                        (cx1 + dx, cy1 - dy),
                    ])
                    nuevos.extend([
                        (cx2 - dx, cy2 - dy),
                        (cx2 + dx, cy2 - dy),
                        (cx2 + dx, cy2 + dy),
                        (cx2 - dx, cy2 + dy),
                    ])
```

```
                    nuevos.extend([
                        (cx3 - dx, cy3 - dy),
                        (cx3 + dx, cy3 - dy),
                        (cx3 + dx, cy3 + dy),
                        (cx3 - dx, cy3 + dy),
                    ])
                    nuevos.extend([
                        (cx4 + dx, cy4 + dy),
                        (cx4 + dx, cy4 - dy),
                        (cx4 - dx, cy4 - dy),
                        (cx4 - dx, cy4 + dy),
                    ])
                    if j == iteraciones-2:
                        tuberia1.extend([
                            (cx1 - dx-radio, cy1 - dy+radio),
                            (cx1 - dx-radio, cy1 + dy+radio),
                            (cx1 + dx+radio, cy1 + dy+radio),
                            (cx1 + dx+radio, cy1 - dy+radio),
                        ])
                        tuberia1.extend([
                            (cx2 - dx-radio, cy2 - dy+radio),
                            (cx2 + dx-radio, cy2 - dy+radio),
                            (cx2 + dx-radio, cy2 + dy-radio),
                            (cx2 - dx-radio, cy2 + dy-radio),
                        ])
                        tuberia1.extend([
                            (cx3 - dx-radio, cy3 - dy+radio),
                            (cx3 + dx-radio, cy3 - dy+radio),
                            (cx3 + dx-radio, cy3 + dy-radio),
                            (cx3 - dx-radio, cy3 + dy-radio),
                        ])
                        tuberia1.extend([
                            (cx4 + dx+radio, cy4 + dy-radio),
                            (cx4 + dx+radio, cy4 - dy-radio),
                            (cx4 - dx-radio, cy4 - dy-radio),
                            (cx4 - dx-radio, cy4 + dy-radio),
                        ])
```

```
                    tuberia2.extend([
                        (cx1 - dx+radio, cy1 - dy-radio),
                        (cx1 - dx+radio, cy1 + dy-radio),
                        (cx1 + dx-radio, cy1 + dy-radio),
                        (cx1 + dx-radio, cy1 - dy-radio),
                    ])
                    tuberia2.extend([
                        (cx2 - dx+radio, cy2 - dy-radio),
                        (cx2 + dx+radio, cy2 - dy-radio),
                        (cx2 + dx+radio, cy2 + dy+radio),
                        (cx2 - dx+radio, cy2 + dy+radio),
                    ])
                    tuberia2.extend([
                        (cx3 - dx+radio, cy3 - dy-radio),
                        (cx3 + dx+radio, cy3 - dy-radio),
                        (cx3 + dx+radio, cy3 + dy+radio),
                        (cx3 - dx+radio, cy3 + dy+radio),
                    ])
                    tuberia2.extend([
                        (cx4 + dx-radio, cy4 + dy+radio),
                        (cx4 + dx-radio, cy4 - dy+radio),
                        (cx4 - dx+radio, cy4 - dy+radio),
                        (cx4 - dx+radio, cy4 + dy+radio),
                    ])
            else:
                nuevos.extend([
                    (cx1 + dx, cy1 + dy),
                    (cx1 + dx, cy1 - dy),
                    (cx1 - dx, cy1 - dy),
                    (cx1 - dx, cy1 + dy),
                ])
                nuevos.extend([
                    (cx2 + dx, cy2 + dy),
                    (cx2 - dx, cy2 + dy),
                    (cx2 - dx, cy2 - dy),
                    (cx2 + dx, cy2 - dy),
                ])
```

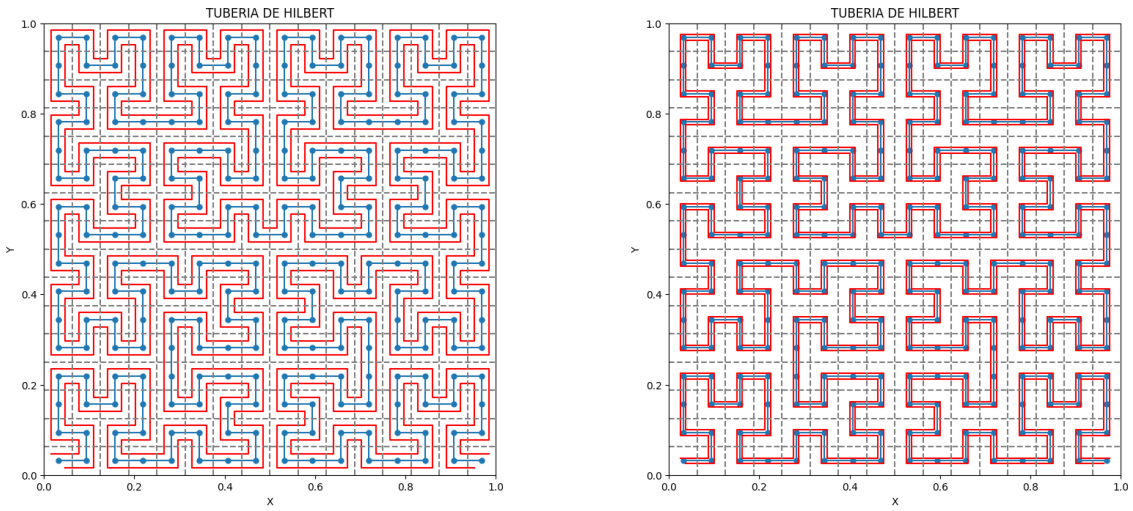
```
                    nuevos.extend([
                        (cx3 + dx, cy3 + dy),
                        (cx3 - dx, cy3 + dy),
                        (cx3 - dx, cy3 - dy),
                        (cx3 + dx, cy3 - dy),
                    ])
                    nuevos.extend([
                        (cx4 - dx, cy4 - dy),
                        (cx4 - dx, cy4 + dy),
                        (cx4 + dx, cy4 + dy),
                        (cx4 + dx, cy4 - dy),
                    ])
                    if j == iteraciones-2:
                        tuberia1.extend([
                            (cx1 + dx+radio, cy1 + dy-radio),
                            (cx1 + dx+radio, cy1 - dy-radio),
                            (cx1 - dx-radio, cy1 - dy-radio),
                            (cx1 - dx-radio, cy1 + dy-radio),
                        ])
                        tuberia1.extend([
                            (cx2 + dx+radio, cy2 + dy-radio),
                            (cx2 - dx+radio, cy2 + dy-radio),
                            (cx2 - dx+radio, cy2 - dy+radio),
                            (cx2 + dx+radio, cy2 - dy+radio),
                        ])
                        tuberia1.extend([
                            (cx3 + dx+radio, cy3 + dy-radio),
                            (cx3 - dx+radio, cy3 + dy-radio),
                            (cx3 - dx+radio, cy3 - dy+radio),
                            (cx3 + dx+radio, cy3 - dy+radio),
                        ])
                        tuberia1.extend([
                            (cx4 - dx-radio, cy4 - dy+radio),
                            (cx4 - dx-radio, cy4 + dy+radio),
                            (cx4 + dx+radio, cy4 + dy+radio),
                            (cx4 + dx+radio, cy4 - dy+radio),
                        ])
```

```
                        tuberia2.extend([
                            (cx1 + dx-radio, cy1 + dy+radio),
                            (cx1 + dx-radio, cy1 - dy+radio),
                            (cx1 - dx+radio, cy1 - dy+radio),
                            (cx1 - dx+radio, cy1 + dy+radio),
                        ])
                        tuberia2.extend([
                            (cx2 + dx-radio, cy2 + dy+radio),
                            (cx2 - dx-radio, cy2 + dy+radio),
                            (cx2 - dx-radio, cy2 - dy-radio),
                            (cx2 + dx-radio, cy2 - dy-radio),
                        ])
                        tuberia2.extend([
                            (cx3 + dx-radio, cy3 + dy+radio),
                            (cx3 - dx-radio, cy3 + dy+radio),
                            (cx3 - dx-radio, cy3 - dy-radio),
                            (cx3 + dx-radio, cy3 - dy-radio),
                        ])
                        tuberia2.extend([
                            (cx4 - dx+radio, cy4 - dy-radio),
                            (cx4 - dx+radio, cy4 + dy-radio),
                            (cx4 + dx-radio, cy4 + dy-radio),
                            (cx4 + dx-radio, cy4 - dy-radio),
                        ])
        centros = nuevos  # Actualizamos la lista completa
    return tuberia1,tuberia2
```

EJECUCIÓN

```
# Parametros
x_max, y_max = 2,5
iteraciones = 5
# Generar centros
centros = centros_hilbert(x_max, y_max, iteraciones)
# Crear figura
plt.figure(figsize=(5, 15))
# Dibujar centros
xs = [c[0] for c in centros]
```

```
ys = [c[1] for c in centros]
plt.scatter(xs, ys, marker='o', s=(9-iteraciones)**2)
plt.plot(xs,ys)
# Dibujar l neas de subdivisi n
step_x = x_max / (2**iteraciones)
step_y = y_max / (2**iteraciones)
# L neas verticales
for i in range(1, 2**iteraciones):
    x = i * step_x
    plt.plot([x, x], [0, y_max], color='gray', linestyle='--')
# L neas horizontales
for i in range(1, 2**iteraciones):
    y = i * step_y
    plt.plot([0, x_max], [y, y], color='gray', linestyle='--')
# Configuraci n del plano
plt.xlim(0, x_max)
plt.ylim(0, y_max)
plt.gca().set_aspect('equal', 'box')
plt.xlabel( X )
plt.ylabel( Y )
plt.title( CURVA DE HILBERT )
plt.show()
```



(a) Tubería Hilbert con porcentaje 50 %

(b) Tubería Hilbert con prcentaje 80 %

Figura A.1: Generar tubería de Hilbert bidimensional

A.5. Libro de arte

En esta sección se encuentran los esquemas de las figuras con los que se han realizado todos los dibujos del trabajo.

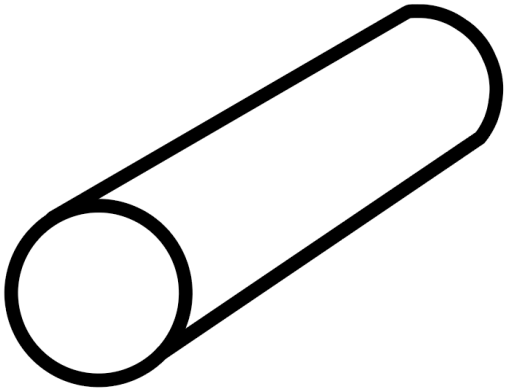


Figura A.2: Tubería cilíndrica. (Rodrigo Arbós. 2026)

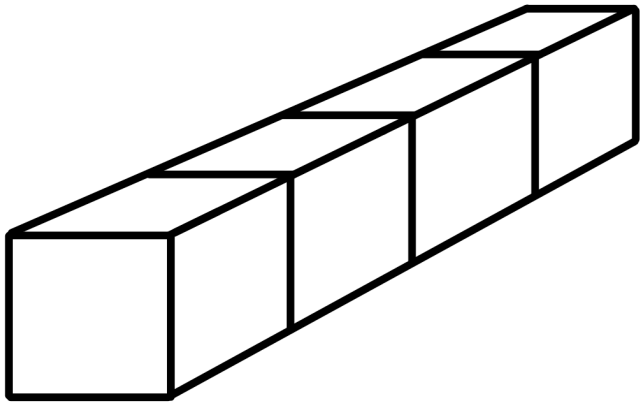


Figura A.3: Tubería rectangular. (Rodrigo Arbós. 2026)

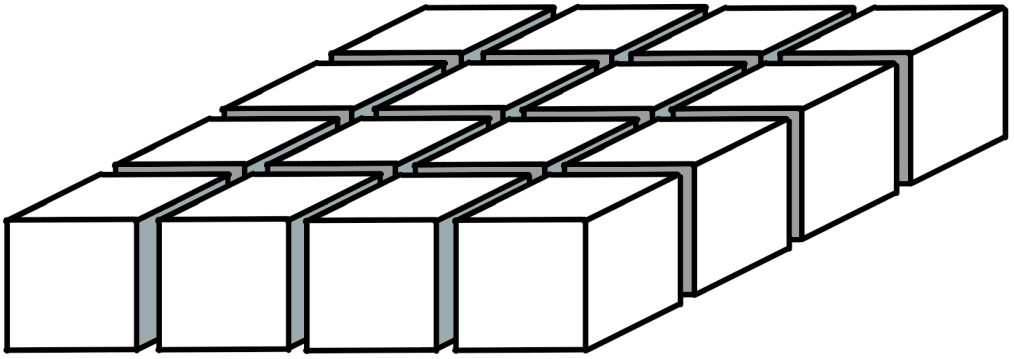


Figura A.4: Separación de una superficie por bloques. (Rodrigo Arbós. 2026)